# Inside the National Research Platform

## How we build it, how we run it, and how you can use it

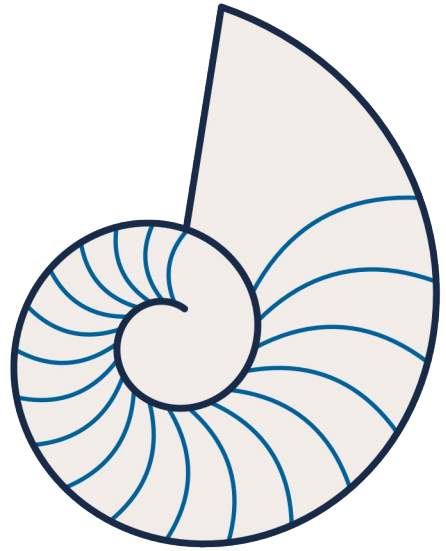
**Daniel Diaz** · San Diego Supercomputer Center, UC San Diego

SCinet · SC26 · Chicago · November 2026



# Agenda

- What is NRP?
- How it's built: the network, storage, hardware, ingress
- How we run it: deploying services, CI in VMs, NetBox, monitoring, perfSONAR, security, sharing GPUs
- AI: the LLM service, from the gateway to the GPUs, and an ops bot built on it
- Using it yourself
- Live demo, then questions



Nautilus: the name of NRP's cluster

Slides and the demo notebook:

*https://training.nrp-nautilus.io/scinet-sc26*

# What NRP Is

# What is NRP?

- One national **Kubernetes** cluster, **Nautilus**, for research and education
- Operated by SDSC (UC San Diego), Nebraska–Lincoln and MGHPCC; funded by NSF

*https://dash.nrp-nautilus.io*




Nodes, GPUs, CPUs (hardware threads) and storage: cluster snapshot, 10 Nov 2026

# What is NRP?

- One national **Kubernetes** cluster, **Nautilus**, for research and education
- Operated by SDSC (UC San Diego), Nebraska–Lincoln and MGHPCC; funded by NSF
- Hardware from 70+ institutions, on research and education networks
  - Internet2 hosts 54 nodes in 51 of its own locations (orange)
- Today:
  - 534 nodes at 129 sites
  - 1,460 GPUs in 33 models; 35,448 CPUs
  - 15 PB of raw Ceph storage

*https://dash.nrp-nautilus.io*




Sites from the NRP dashboard, 10 Nov 2026
Not drawn: 8 sites in Hawaii, Guam, Korea, Brazil and Europe

Nodes, GPUs, CPUs (hardware threads) and storage: cluster snapshot, 10 Nov 2026

# What is NRP?

- One national **Kubernetes** cluster, **Nautilus**, for research and education
- Operated by SDSC (UC San Diego), Nebraska–Lincoln and MGHPCC; funded by NSF
- Hardware from 70+ institutions, on research and education networks
  - Internet2 hosts 54 nodes in 51 of its own locations (orange)
- Today:
  - 534 nodes at 129 sites
  - 1,460 GPUs in 33 models; 35,448 CPUs
  - 15 PB of raw Ceph storage
- Free for non-commercial research and education
- One team runs it from the OS up; campuses keep priority on what they contribute

https://dash.nrp-nautilus.io

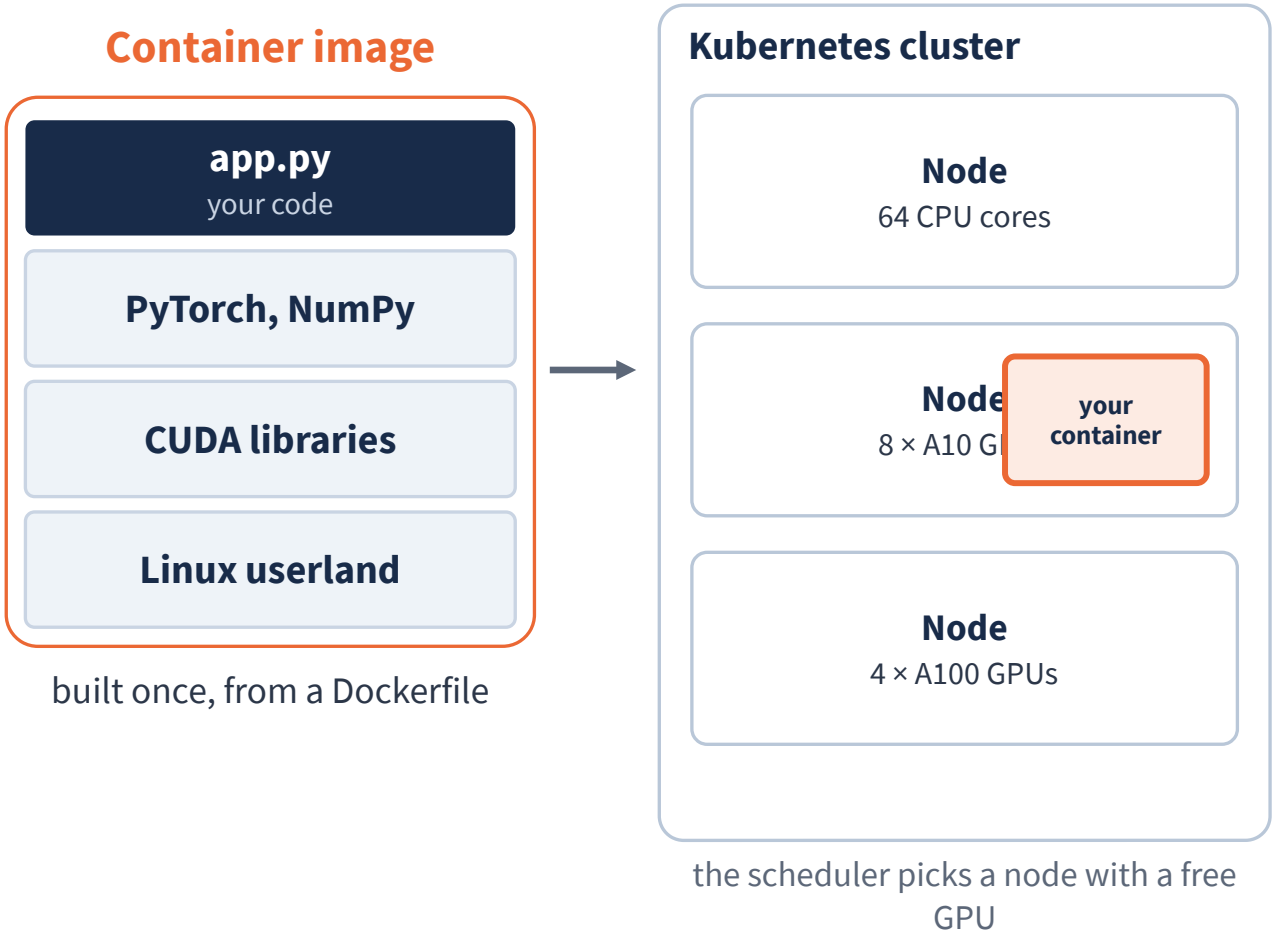



Sites from the NRP dashboard, 10 Nov 2026
Not drawn: 8 sites in Hawaii, Guam, Korea, Brazil and Europe

Nodes, GPUs, CPUs (hardware threads) and storage: cluster snapshot, 10 Nov 2026

# Containers and Kubernetes

- **Container image:** an app and everything it needs, built once from a Dockerfile; runs the same anywhere
- **Kubernetes** runs containers across many machines: you declare what you want (an image, CPUs, memory, a GPU), the **scheduler** picks a **node** that has it, and controllers keep it that way
- A running container is a **pod**. Pods, Jobs, Deployments and Services live in a **namespace**: one per lab, course or service
- NRP is one Kubernetes cluster: 534 nodes



# From PRP to NRP

## 2015

### Pacific Research Platform

Data transfer nodes (FIONAs) in campus Science DMZs, on CENIC and Pacific Wave

OAC-1541349

## 2018

### Toward a National Research Platform

The same model, beyond the West Coast

OAC-1826967

## 2021

### National Research Platform

An NSF Category II resource: GPUs, storage and services for research and education

OAC-2112167

## 2026

### Today

One Kubernetes cluster, Nautilus: 534 nodes at 129 sites, and AI models as a service

Under each milestone: its NSF award.

# NRP in Numbers

**534**
nodes

**129**
sites on the dashboard

**1,460**
GPUs in 33 models

**35,448**
CPUs (hardware threads)

**14.9 PB**
raw Ceph in 9 clusters

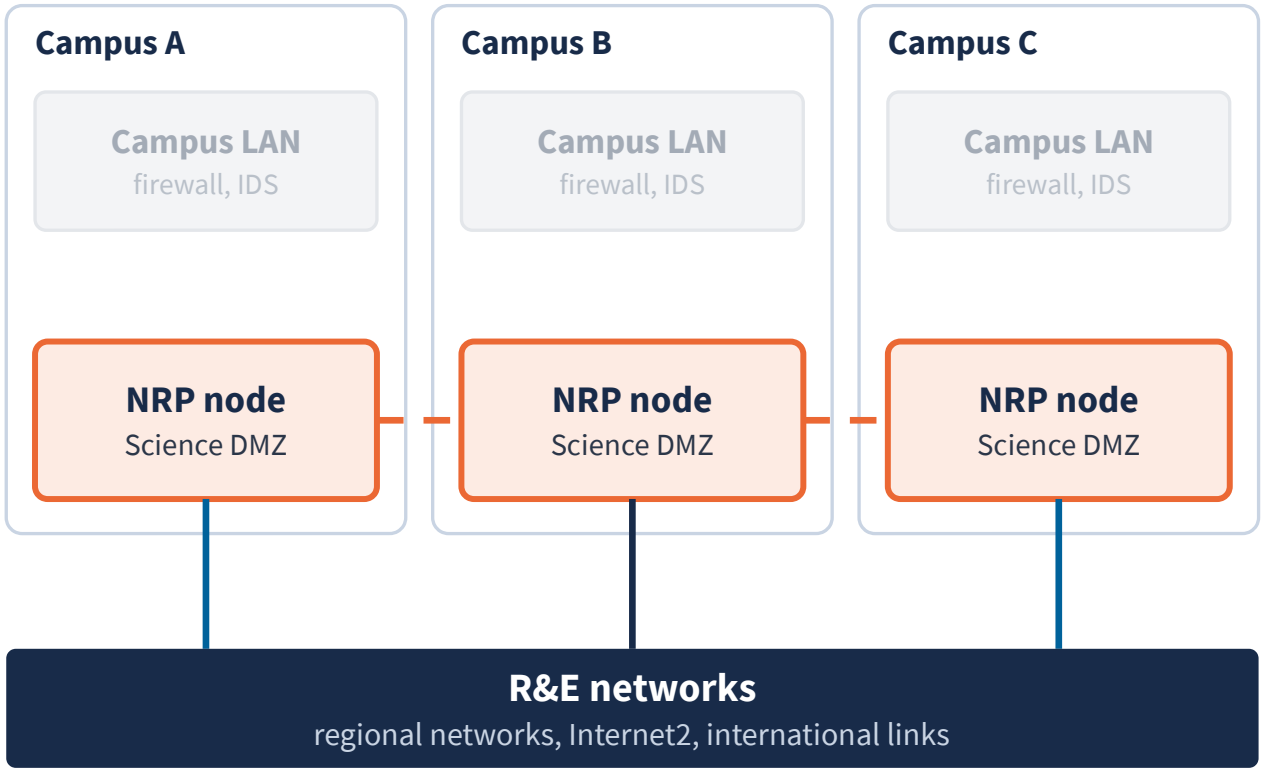
**2,334**
namespaces: labs, courses, services

Cluster snapshot, 10 Nov 2026 (Kubernetes v1.34.11). Sites: the NRP dashboard.

# How It's Built

# The Network Underneath

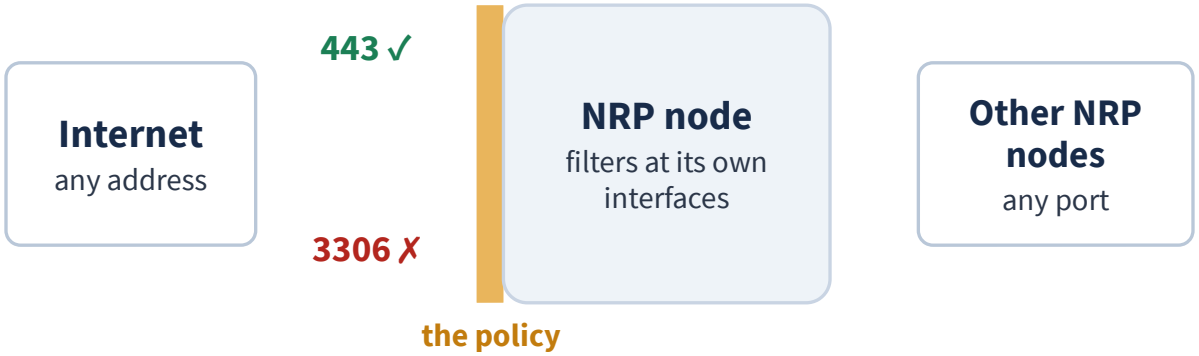
- Nodes sit in campus **Science DMZs**, outside campus firewalls, at 10–100 Gb/s
- **Jumbo frames:** MTU 9000 between all nodes
- Regional networks and Internet2 carry the traffic
- **Calico** builds the pod network: a **VXLAN** overlay, IPv4 and IPv6 (309 nodes dual-stack)
- Pod to pod, the packet is wrapped in UDP between the two nodes' addresses and crosses the real network




Orange dot: what the pods see, one hop on the overlay. Gold dot: what the network carries, the same packet inside VXLAN, node to node.

# Policy Lives in the Cluster

- Nodes sit outside campus firewalls, so the cluster filters traffic **at each node** itself
- One **Calico GlobalNetworkPolicy** for every node, written once and stored in the cluster
- From other NRP nodes: everything is allowed
- From the internet: only the ports in the table
- A new node is covered the moment it joins; campuses keep no per-node firewall rules, and turn local iptables and firewalld off
- Why: nodes come and go every week, faster than any campus change process




| From the internet | Why |
|---|---|
| `22/tcp` | SSH, for NRP operators |
| `443/tcp` | HTTPS; open on every node |
| `ICMP` | ping, and path MTU discovery for jumbo frames |
| `8443/tcp` | object reads over HTTPS and XRootD (data caches) |
| `8444/tcp` | Pelican tokens, director health checks, metrics |
| `50000–51000/tcp` | GridFTP data channels |

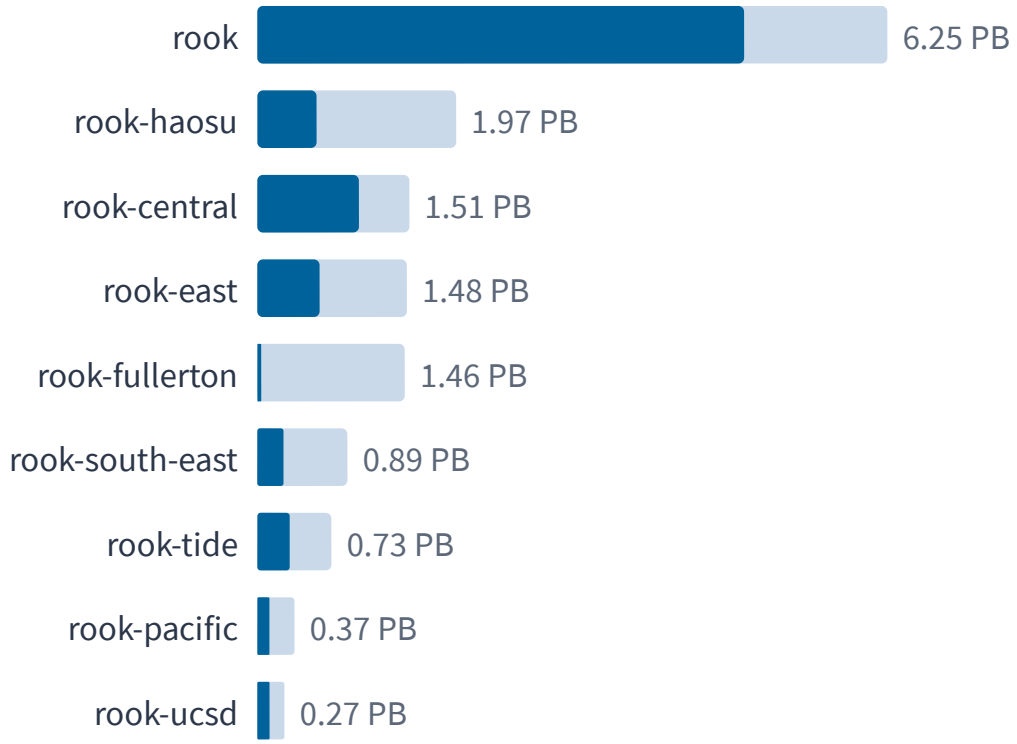
The last three only on nodes that run a data cache, an origin or a Globus endpoint.

*https://nrp.ai/documentation/admindocs/participating/network/*

# Regions and Storage

- Every node is labeled with its **region**, **zone** (143 zones) and **site**; workloads spread out or stay near their data
- **Rook** runs 9 **Ceph** clusters, one per region or large site: 14.9 PB raw
- CephFS for shared files, RBD for block, S3 for objects
- **Linstor** (DRBD) for fast replicated block storage; **CVMFS** for software; local NVMe for scratch
- Pick the storage in the region where you compute

**Ceph clusters: raw capacity, and used**




From the CephCluster resources, 10 Nov 2026

# More Than GPUs

## Hardware beyond x86 and NVIDIA

**NVIDIA GPUs**
1,460 in 33 models, GTX 1080 Ti to H200

**AMD GPUs**
ROCm device plugin

**Qualcomm Cloud AI 100**
inference cards, requested like a GPU

**Xilinx Alveo FPGAs**
Vivado and Vitis

**ESnet SmartNIC**
FPGA network cards you program in P4

**ARM64 nodes**
a second CPU architecture (3 nodes): needs multi-arch images

## Runtimes and networking

**Containers**
runc, plus the NVIDIA runtime

**Kata VMs**
a lightweight VM per pod

**Sysbox**
system containers: Docker inside a pod

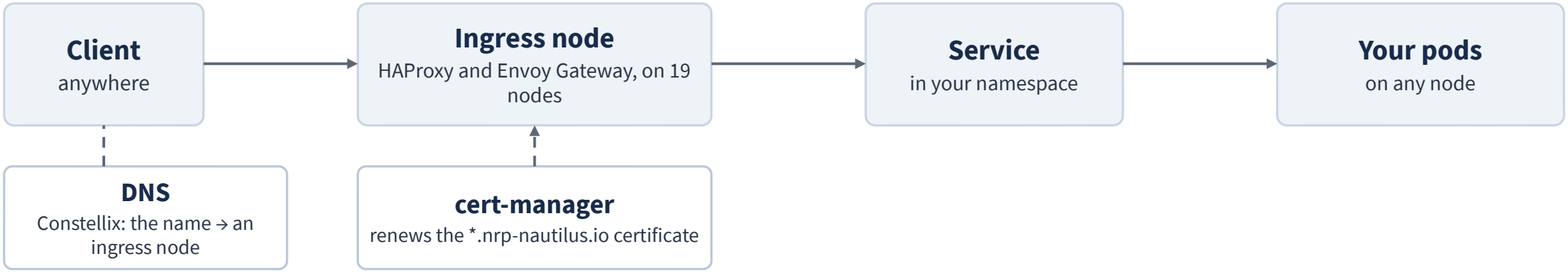
**KubeVirt**
full VMs: Ubuntu, Windows, Talos

**Multus and SR-IOV**
extra NICs in a pod, for network research

**Confidential VMs**
Kata with AMD SNP or Intel TDX, GPUs included

Runtime classes on the cluster: 29, most of them Kata variants (QEMU, Cloud Hypervisor, Firecracker, Dragonball).

# Getting Traffic In

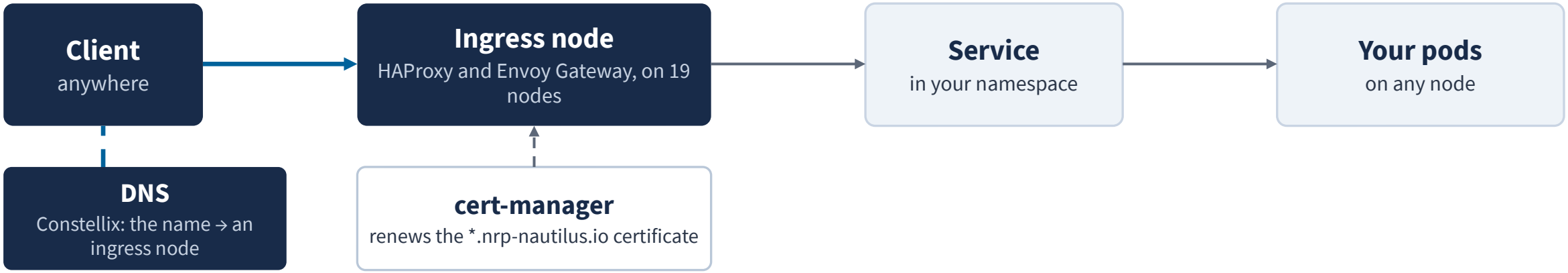



## A request from the internet to a pod

Web traffic comes in through ingress nodes; anything else gets its own address.

1,006 Ingress objects (HAProxy) and 33 Gateway API routes (Envoy Gateway); users create them in their own namespace.

# Getting Traffic In

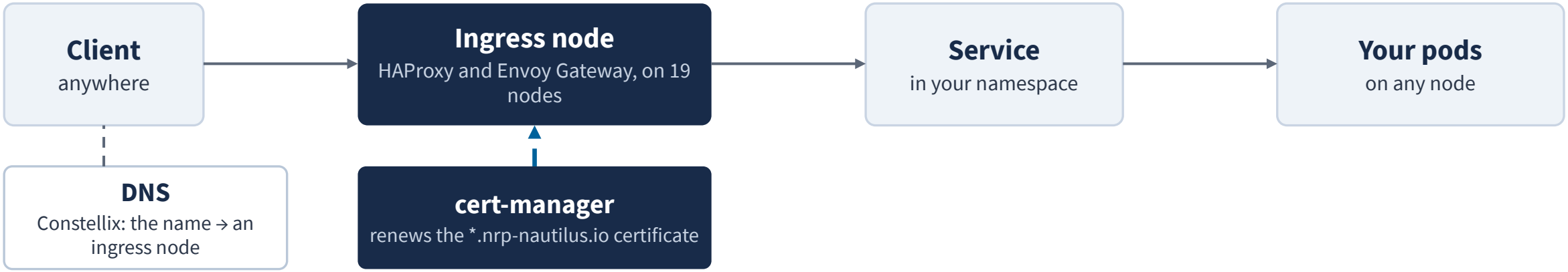



## 1. Find it

A name under nrp-nautilus.io resolves to one of the 19 ingress nodes, which sit at sites in the US, Europe and Korea. The client connects to it on port 443.

1,006 Ingress objects (HAProxy) and 33 Gateway API routes (Envoy Gateway); users create them in their own namespace.

# Getting Traffic In

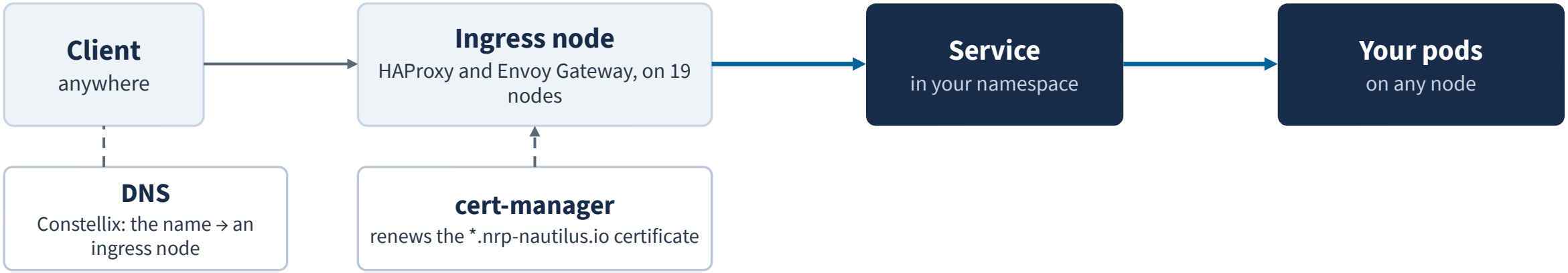



## 2. Decrypt

HAProxy on that node ends TLS with the *.nrp-nautilus.io certificate that cert-manager renews from Let's Encrypt, reads the host name, and finds the matching Ingress rule.

1,006 Ingress objects (HAProxy) and 33 Gateway API routes (Envoy Gateway); users create them in their own namespace.

# Getting Traffic In

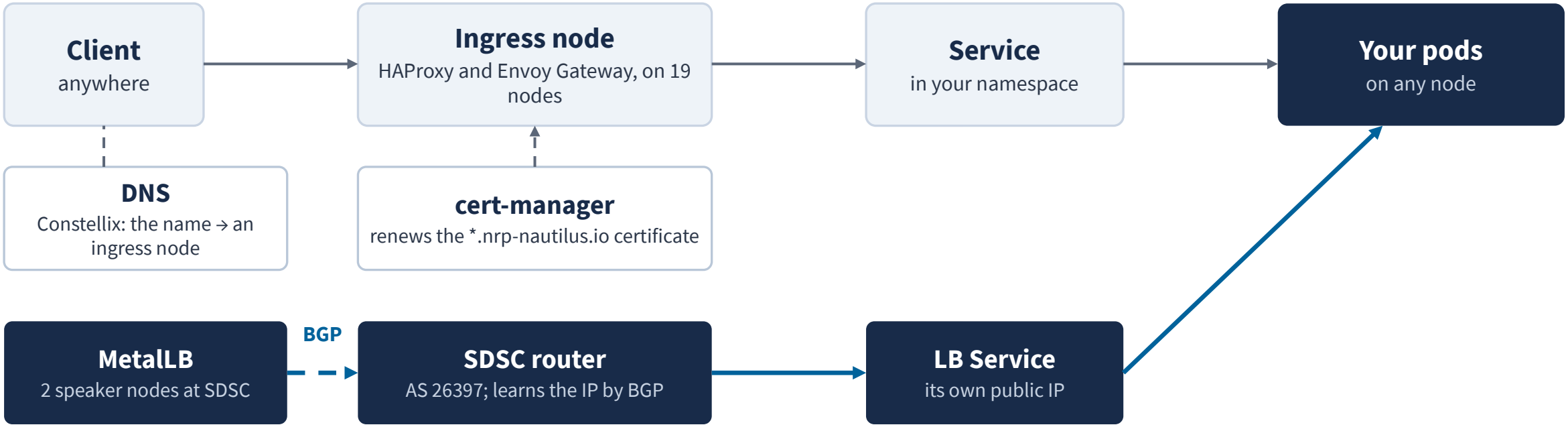



## 3. Deliver

The rule names a Service in the owner's namespace. The Service spreads requests over its pods, wherever they run, across the overlay.

1,006 Ingress objects (HAProxy) and 33 Gateway API routes (Envoy Gateway); users create them in their own namespace.

# Getting Traffic In




## Not HTTP?

A LoadBalancer Service gets its own public IP. MetalLB, on two nodes at SDSC, announces it to SDSC's router over BGP, for IPv4 and IPv6.

1,006 Ingress objects (HAProxy) and 33 Gateway API routes (Envoy Gateway); users create them in their own namespace.

# How We Run It

# How Services Get Deployed




**Who uses it, and for what**

**Admins:** platform services, such as Falco, NetBox, Grafana and GitLab itself

**Admins and users:** the most common path: build an image, roll it out to a namespace

**Admins:** some services, like the LLM servers. **Users:** most work in their own namespace

Next: Argo CD up close, then CI. Whichever way it comes, a service ends up as Kubernetes objects in a namespace.

# How Services Get Deployed




**Who uses it, and for what**

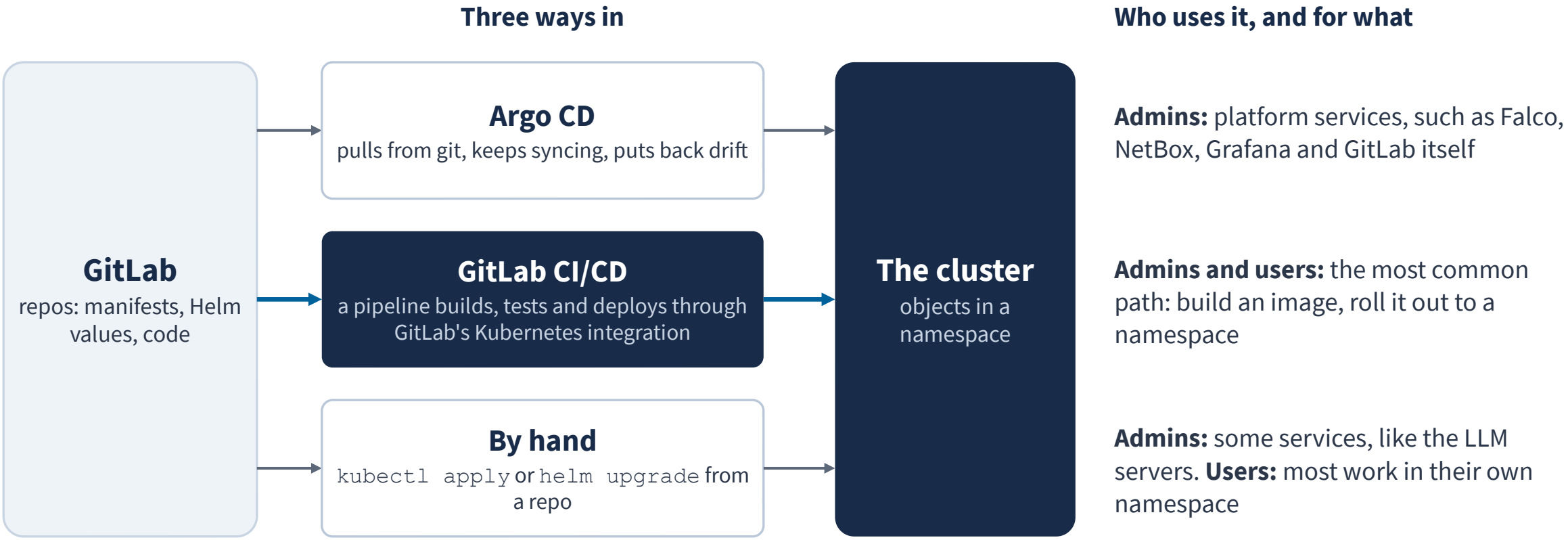
**Admins:** platform services, such as Falco, NetBox, Grafana and GitLab itself

**Admins and users:** the most common path: build an image, roll it out to a namespace

**Admins:** some services, like the LLM servers. **Users:** most work in their own namespace

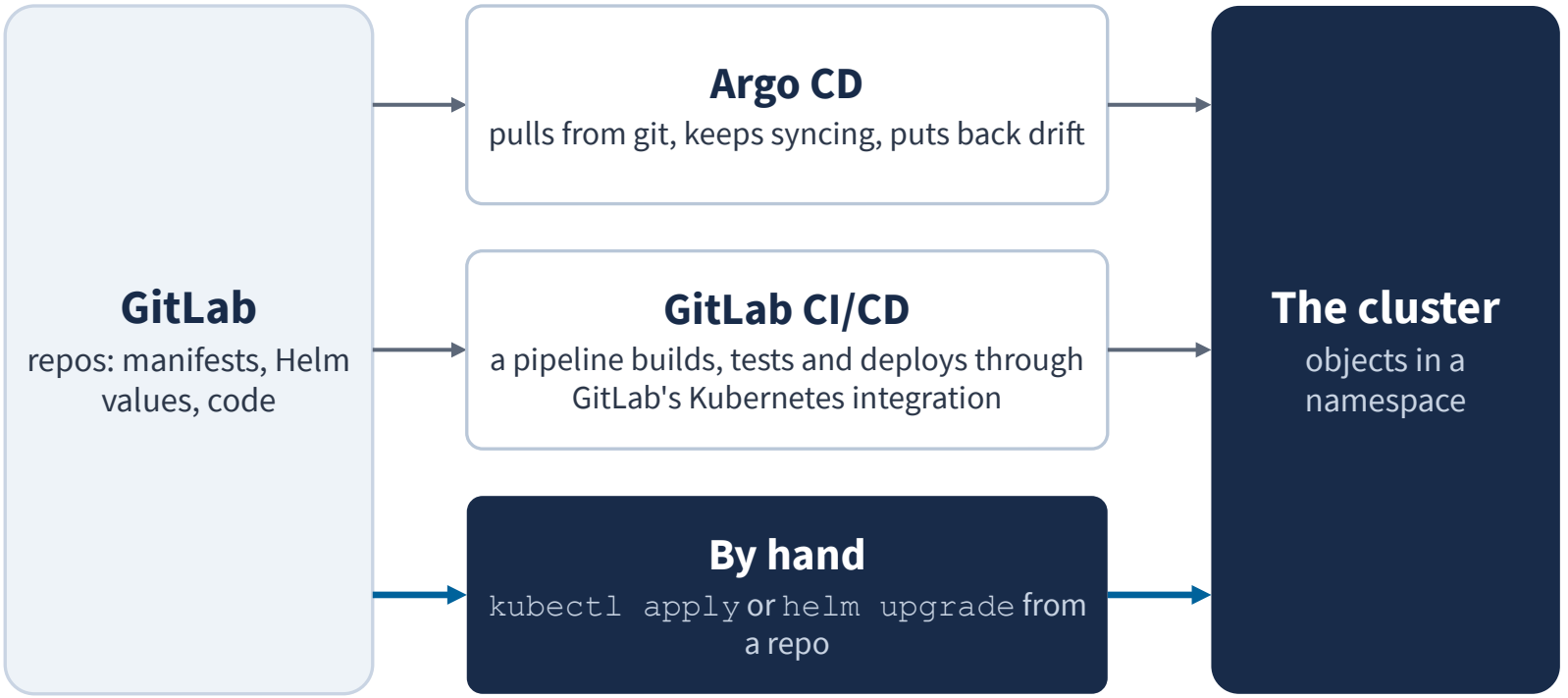
Next: Argo CD up close, then CI. Whichever way it comes, a service ends up as Kubernetes objects in a namespace.

# How Services Get Deployed




Next: Argo CD up close, then CI. Whichever way it comes, a service ends up as Kubernetes objects in a namespace.

# How Services Get Deployed




**Who uses it, and for what**

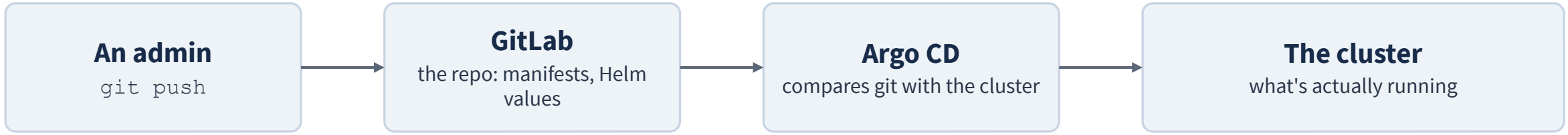
**Admins:** platform services, such as Falco, NetBox, Grafana and GitLab itself

**Admins and users:** the most common path: build an image, roll it out to a namespace

**Admins:** some services, like the LLM servers. **Users:** most work in their own namespace

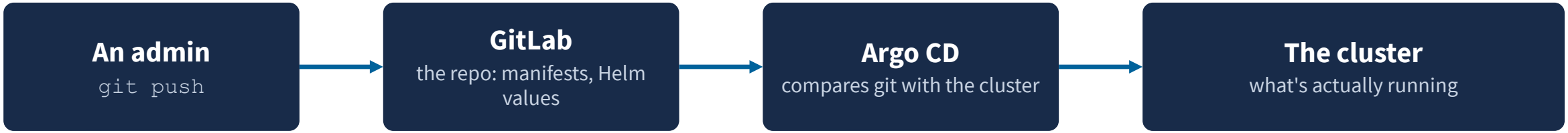
Next: Argo CD up close, then CI. Whichever way it comes, a service ends up as Kubernetes objects in a namespace.

# Git Is the Source of Truth



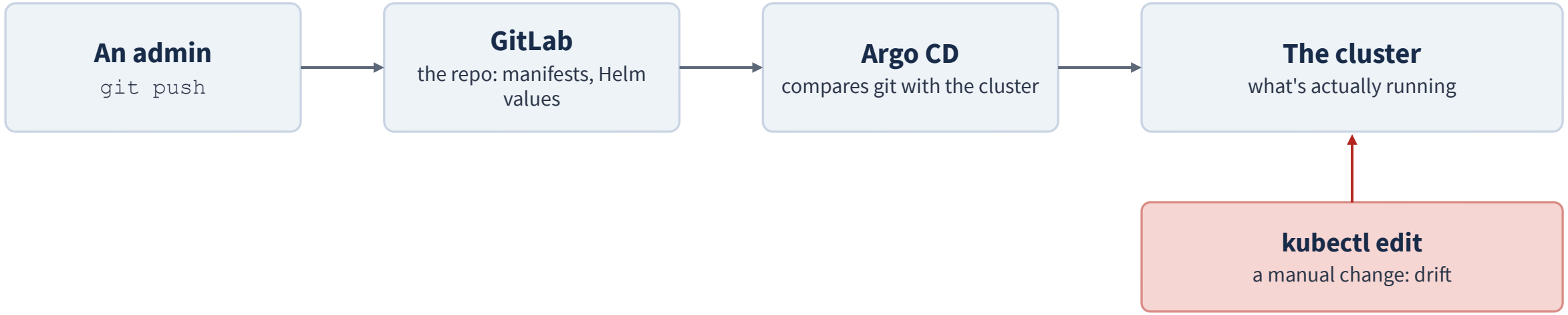

- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Argo CD is one of the three ways in: other services deploy from GitLab CI, or by hand from their repos

# Git Is the Source of Truth



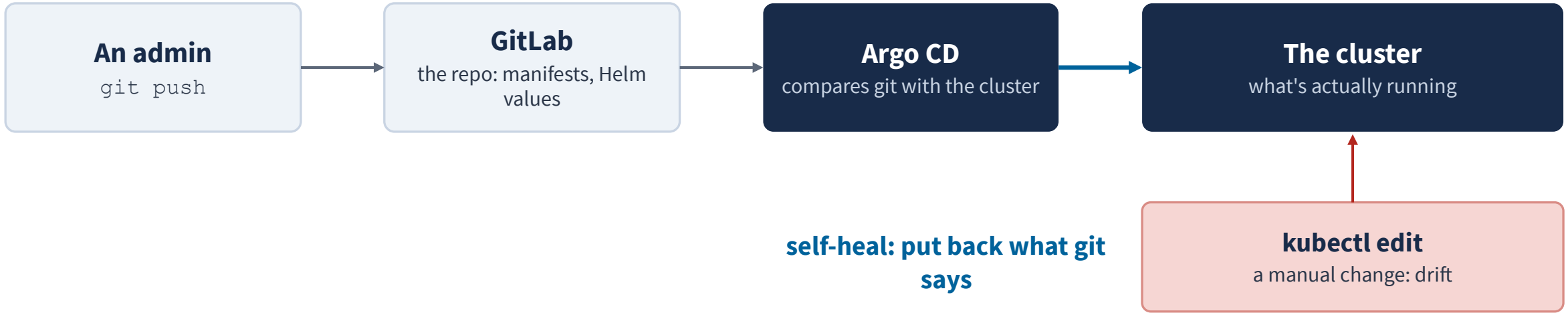

- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Argo CD is one of the three ways in: other services deploy from GitLab CI, or by hand from their repos

# Git Is the Source of Truth



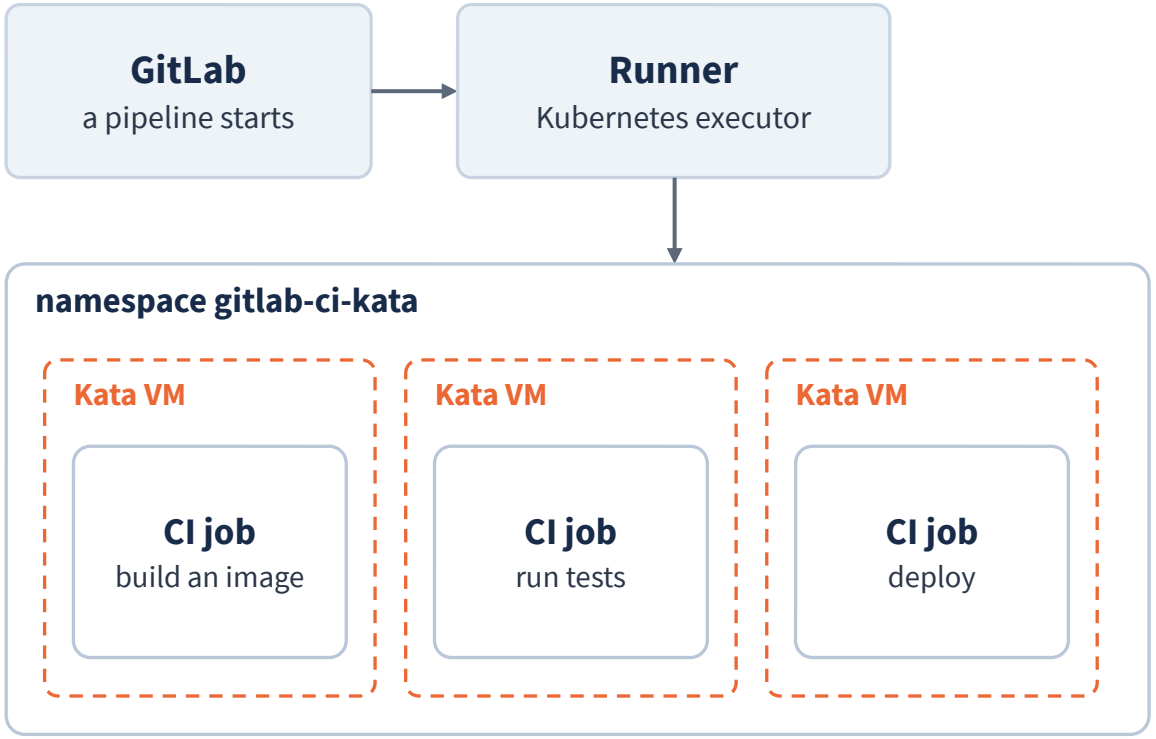

- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Argo CD is one of the three ways in: other services deploy from GitLab CI, or by hand from their repos

# Git Is the Source of Truth




- Self-hosted **GitLab** 19.5.4: code, issues, a container registry, CI
- **Argo CD** syncs 20 applications, GitLab itself among them
- **Self-heal:** a manual change is reverted to what's in git; history and review come free
- Argo CD is one of the three ways in: other services deploy from GitLab CI, or by hand from their repos

# CI in a VM per Job




The default runner's config (from the cluster)

```
[[runners]]
  [runners.kubernetes]
    namespace = "gitlab-ci-kata"
    runtime_class_name = "kata"
    privileged = false
    image = "alpine:3.22"
    pull_policy = ["always"]
    cpu_limit = "4"
    cpu_limit_overwrite_max_allowed = "12"
    memory_limit = "8Gi"
    memory_limit_overwrite_max_allowed = "50Gi"
```

- The shared runners start each CI job as a pod in its own lightweight VM: even a privileged build can't reach the node
- 4 runner flavors (backup slide); images go to **gitlab-registry.nrp-nautilus.io** and are pulled from inside the cluster

# Build and Deploy Your Own

.gitlab-ci.yml (from the NRP docs)

```
image: ghcr.io/osscontainertools/kaniko:debug

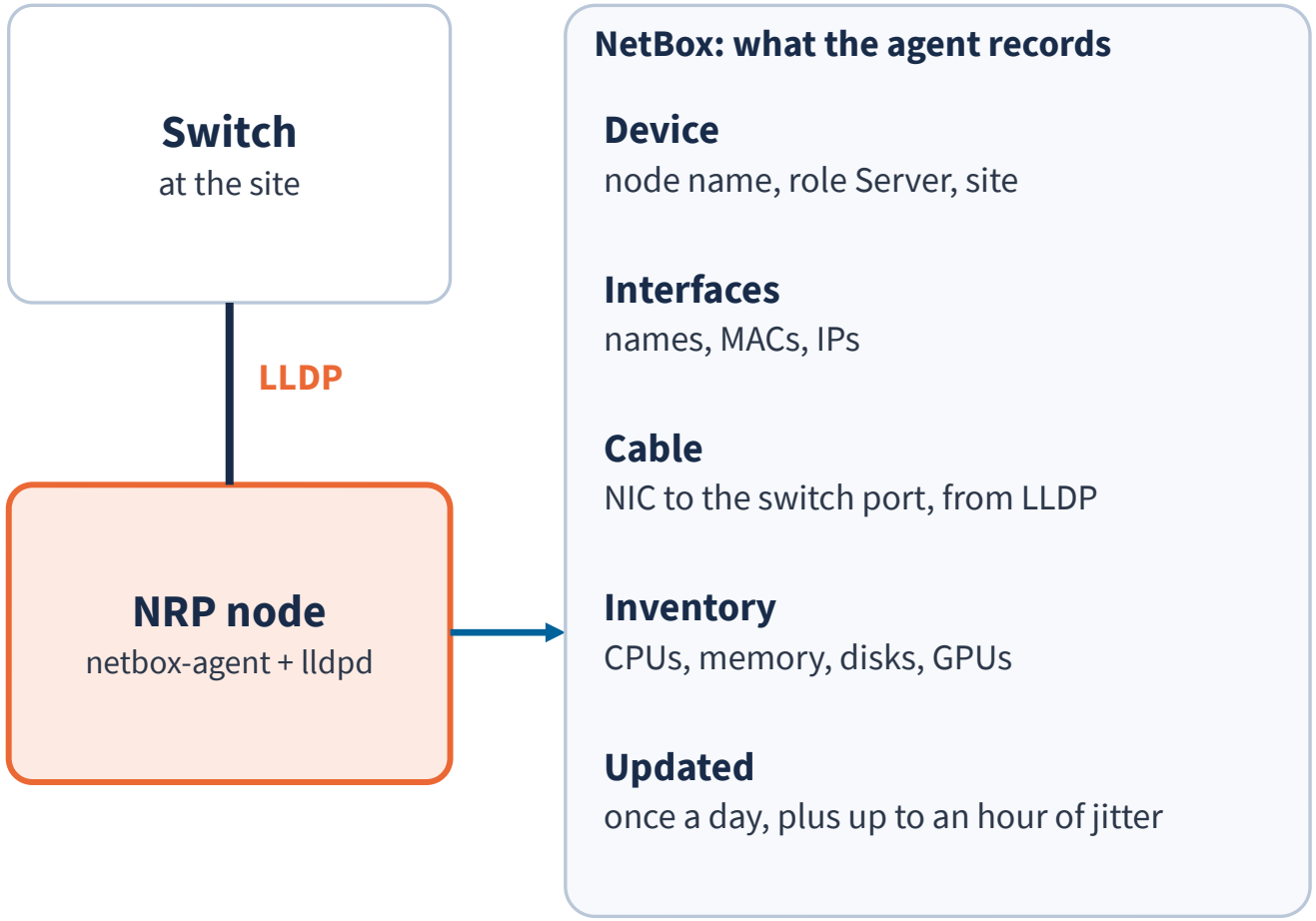
build-and-push-job:
  stage: build
  variables:
    GODEBUG: "http2client=0"
  script:
    - echo "{\"auths\":{\"$CI_REGISTRY\":{\"username\":
        \"$CI_REGISTRY_USER\",\"password\":\"$CI_REGISTRY_PASSWORD\"}}}"
        > /kaniko/.docker/config.json
    - /kaniko/executor --cache=true --push-retry=10
        --context $CI_PROJECT_DIR
        --dockerfile $CI_PROJECT_DIR/Dockerfile
        --destination $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
```

*https://nrp.ai/documentation/userdocs/development/gitlab/*

- Any NRP user can create projects at **gitlab.nrp-nautilus.io**
- Kaniko builds an image with no Docker daemon
- The registry is in the cluster: pulls are fast
- Deploy to your namespace through GitLab's Kubernetes integration: the CI/CD path from before
- Build multi-arch images for the ARM64 nodes

# NetBox Knows Every Node

- **NetBox 4**, deployed by Argo CD from the NetBox Labs Helm chart
- **netbox-agent** on every node (529 pods): privileged, host network
- **lldpd** learns the switch and port each NIC is cabled to
- The site comes from the node's `netbox.io/site` label
- Alongside: Diode for ingest, and a Nornir pod for automation



# Watching the Cluster

## Sources

**node-exporter**
on every node

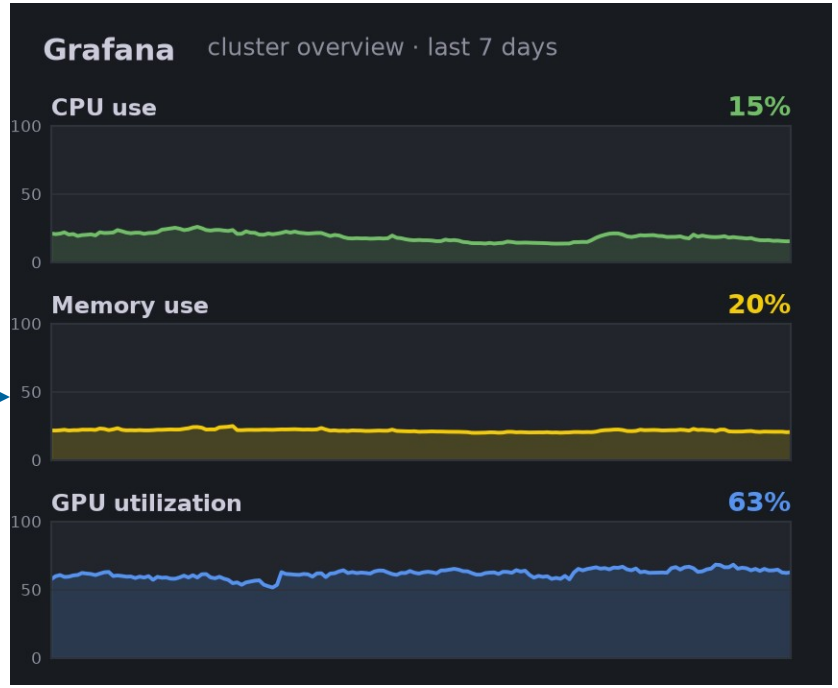
**kube-state-metrics**
objects and their state

**DCGM**
GPU use, memory, power

**Apps**
Ceph, the ingress, every service

**Prometheus**
2 replicas, run by the Prometheus operator

**Thanos**
querier (14 replicas), store, compactor




**Alertmanager**
3 replicas; routes alerts

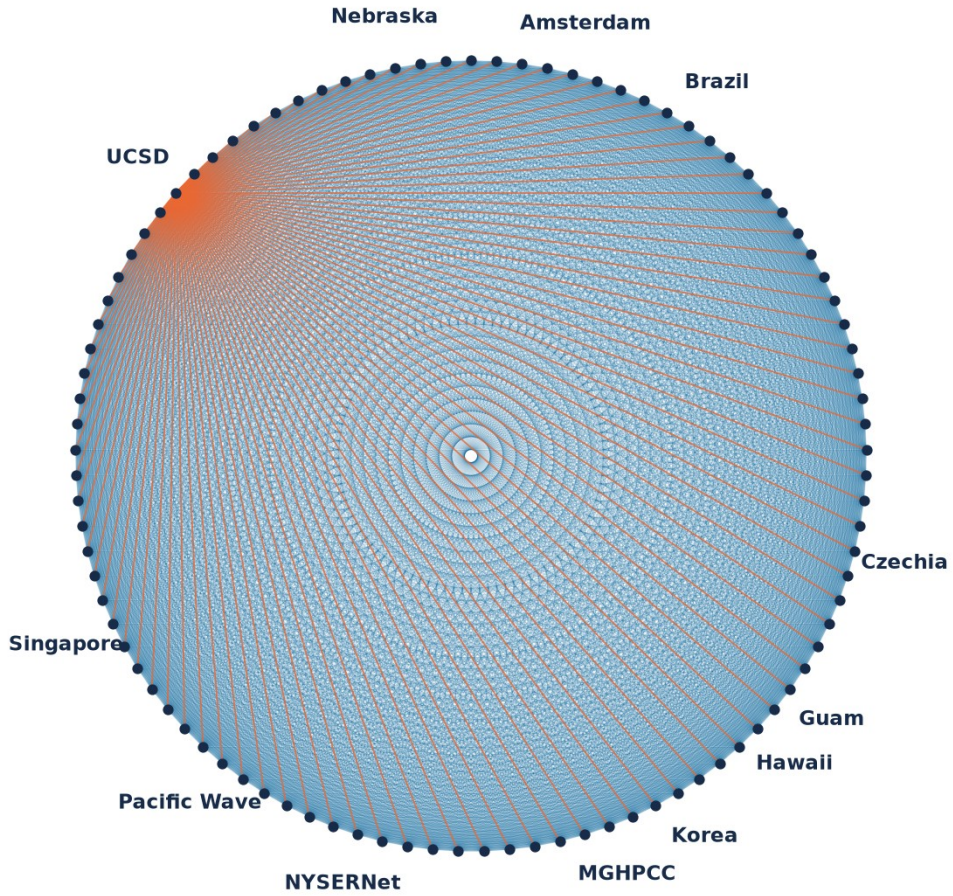
**Public dashboards**
nrp.ai/viz

From Thanos, hourly, 4–11 Nov 2026: CPU and memory averaged over nodes, GPU over GPUs

Logs: Elastic collects syslog from every node, SSH logins included.

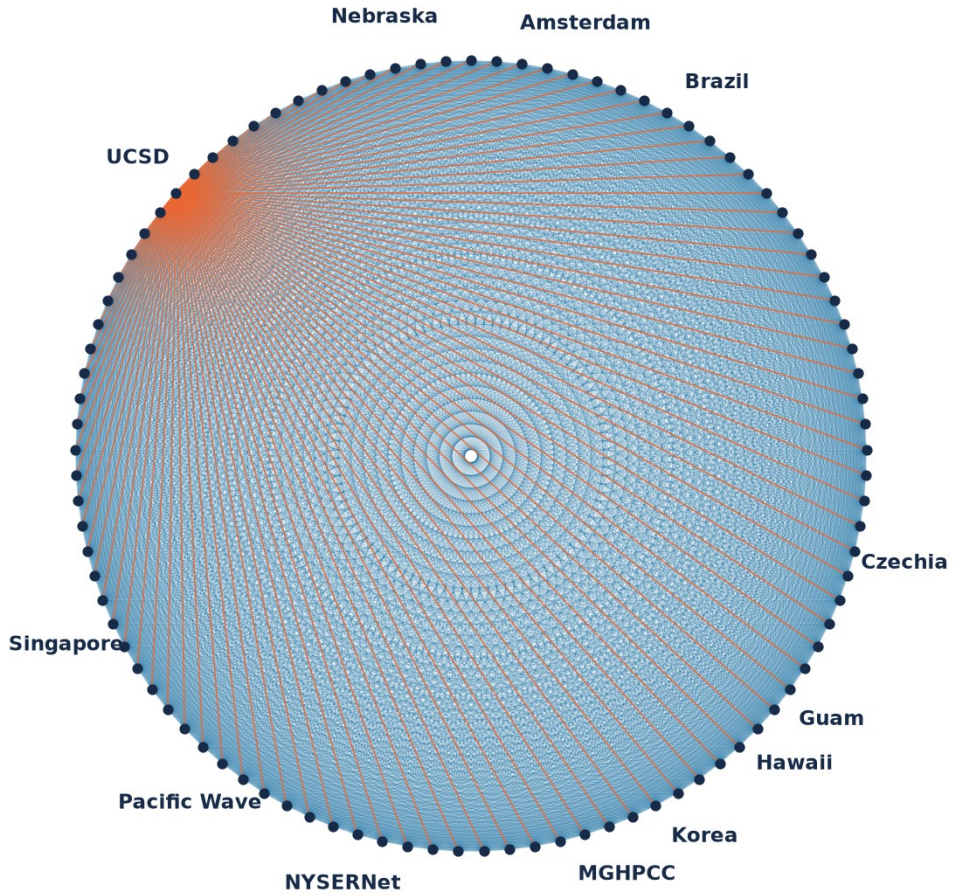
# Measuring the Network

- Two **perfSONAR testpoints** on every node (526 each): one on the host network, one in the pod network
- A **mesh config** says who tests whom; each testpoint's **pScheduler** runs its share



# Measuring the Network

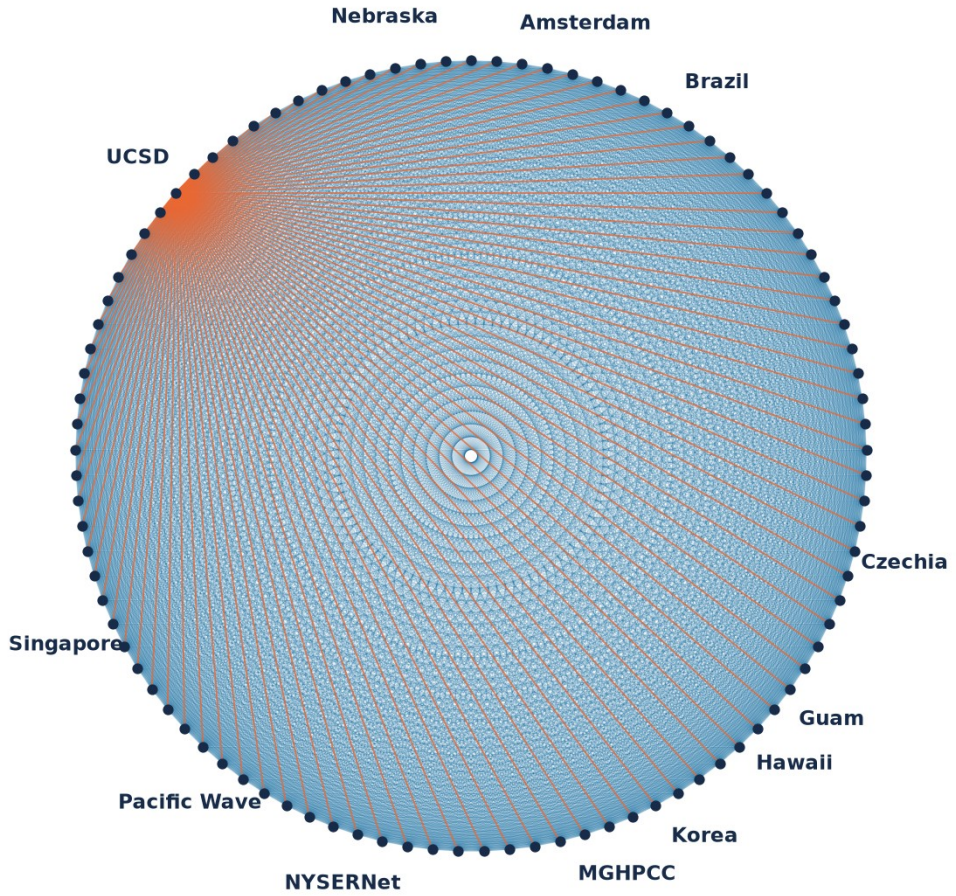
- Two **perfSONAR testpoints** on every node (526 each): one on the host network, one in the pod network
- A **mesh config** says who tests whom; each testpoint's **pScheduler** runs its share
- **All-to-all** between one node per site (97 sites): one-way latency and loss nonstop, throughput every 18 h, traceroute every 6 h




Orange: one site's tests to every other site

# Measuring the Network

- Two **perfSONAR testpoints** on every node (526 each): one on the host network, one in the pod network
- A **mesh config** says who tests whom; each testpoint's **pScheduler** runs its share
- **All-to-all** between one node per site (97 sites): one-way latency and loss nonstop, throughput every 18 h, traceroute every 6 h




Every site tests every other: 97 × 96 directed pairs

# Measuring the Network

- Two **perfSONAR testpoints** on every node (526 each): one on the host network, one in the pod network
- A **mesh config** says who tests whom; each testpoint's **pScheduler** runs its share
- **All-to-all** between one node per site (97 sites): one-way latency and loss nonstop, throughput every 18 h, traceroute every 6 h
- Results go to an **archive**; **MaDDash** draws them as a grid at perfsonar.nrp-nautilus.io
- Host network vs pod network: is a slow path the network, or the overlay?




MaDDash packet-loss grid, 11 Nov 2026: green ≤ 0.001%, yellow more, red ≥ 0.1%, gray no data yet

# Security

## Identity

- Authentik
- CILogon: federated campus login
- Groups drive namespace access

e.g. a new login sees no namespaces until a namespace admin adds it

## Detect

- Falco: runtime alerts
- Falco Talon: automated response
- Wazuh; Elastic syslog, SSH logins included

e.g. a pod runs a crypto miner or calls a mining pool: Falco alerts, Talon kills the pod

## Prevent

- Gatekeeper admission policies
- Kata VMs for CI and untrusted builds
- Central Calico policy
- CVE mitigations rolled out by Argo CD

e.g. a pod that asks to be privileged, or sets CUDA_VISIBLE_DEVICES to reach GPUs it wasn't given, is rejected

## Audit

- IPMI scanner: alerts if a BMC is reachable from outside
- JupyterHub scanner: no anonymous hubs

e.g. a node's BMC answers from the internet: operators get an alert

# Routine Care

| Tool | What it does |
|---|---|
| `kured` | Rolling reboots after kernel updates, a few nodes at a time |
| `self-node-remediation` | Fences and recovers nodes that stop responding |
| `node-label-monitor` | Watches node labels the scheduler depends on |
| `GPU operator, power management` | Drivers, and GPU power tracking |
| `Nagios NRPE` | Host checks |
| `Upgrade runbooks` | Kubernetes, Calico, Rook, GitLab, JupyterHub, Prometheus ... in the admin docs |

*https://nrp.ai/documentation/admindocs/upgrades/intro/*

# Sharing GPUs Fairly

- **Reserved nodes:** on request, admins taint a contributor's nodes; only pods that **tolerate** the taint, the owner's, run there
  - Today: 119 nodes reserved for 23 groups




A pod without the toleration can't land on the reserved node, even on an idle GPU.

# Sharing GPUs Fairly

- **Reserved nodes:** on request, admins taint a contributor's nodes; only pods that **tolerate** the taint, the owner's, run there
  - Today: 119 nodes reserved for 23 groups
- **Shared nodes:** idle GPUs go to **opportunistic** pods, the lowest priority class, e.g. OSG jobs for IceCube and LIGO
  - 232 opportunistic pods running, 208 of them from OSG




On a shared node, opportunistic pods fill the idle GPUs.

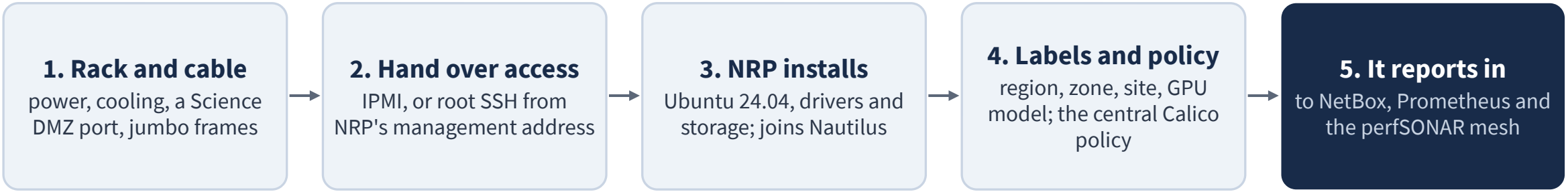
# Sharing GPUs Fairly

- **Reserved nodes:** on request, admins taint a contributor's nodes; only pods that **tolerate** the taint, the owner's, run there
  - Today: 119 nodes reserved for 23 groups
- **Shared nodes:** idle GPUs go to **opportunistic** pods, the lowest priority class, e.g. OSG jobs for IceCube and LIGO
  - 232 opportunistic pods running, 208 of them from OSG
- When anyone needs that GPU, the opportunistic pod is **preempted**: it must checkpoint, or just restart




A normal pod arrives: the scheduler preempts an opportunistic pod to make room.

# How a Node Joins




## Host site

- Keeps an on-site contact for hands-on work
- Sends maintenance notices; forwards warranty tickets

## NRP

- Patching, packages and host hardening
- 24/7 monitoring and alerting; incident handling from the OS up

**Then it's schedulable: shared with everyone, or reserved for your group if you ask, with a taint only your pods tolerate.**

*https://nrp.ai/documentation/admindocs/participating/new-contributor-guide/*

# AI on NRP

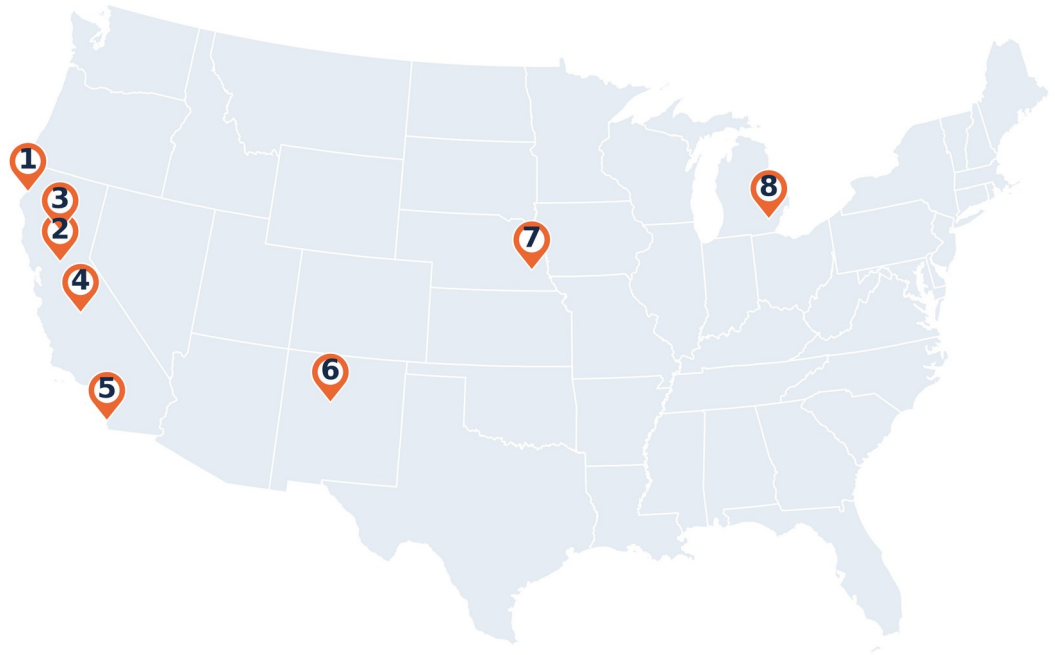
# The LLM Service

- OpenAI API: `ellm.nrp-nautilus.io/v1`
- Anthropic API, for Claude Code and similar tools: `ellm.nrp-nautilus.io/anthropic`
- 25 model names, the same on both
- 83 GPUs serve them, from A10s to H200s, at 9 sites
- About 7.1 billion tokens a day (September 2026)
- Free for research and education, with a token from nrp.ai/llmtoken

| Model name | Serving on |
|---|---|
| **deepseek-v4-flash** | 16 × H100 |
| **glm-5** | 8 × H200, 8 × H100 |
| **kimi** | 8 × A100 |
| **minimax** | 8 × A100 |
| **qwen3** | 8 × A100 |
| **qwen3-small** | 4 × A6000, 2 × H200 |
| **gpt-oss** | 2 × A6000 |
| **gemma** | 4 × A100, 1 × A6000 |
| **gemma-small** | 9 × A10 |
| **gemma-small-e4b** | 1 × A6000 |
| **qwen3-embedding** | 2 × L40, 2 × V100 |

Model-server pods by GPU, 10 Nov 2026. The catalog changes.

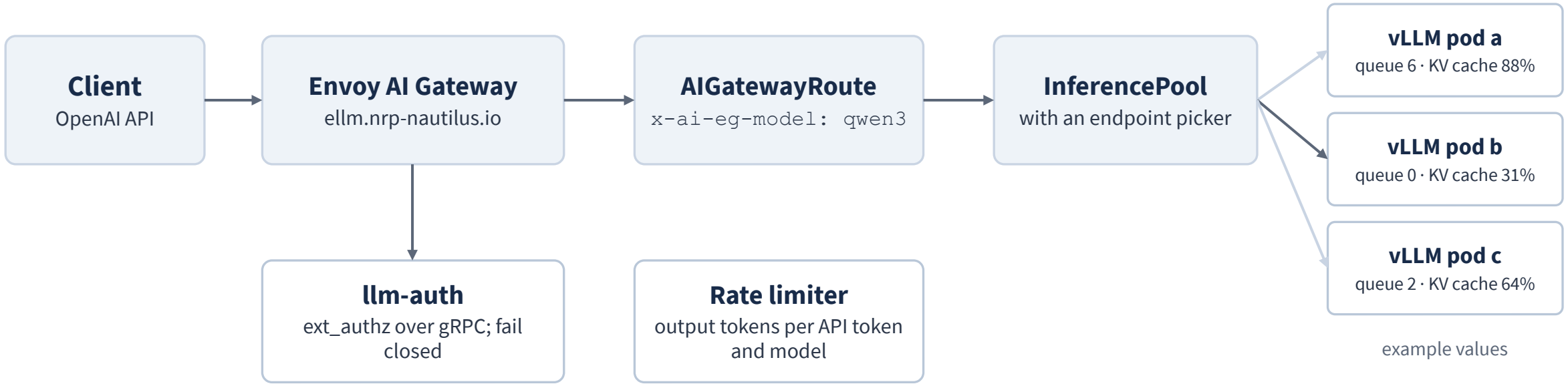
# Where the Models Run




| # | Site | Models | GPUs |
|---|---|---|---|
| 1 | Cal Poly Humboldt | qwen3-embedding | 2 V100 |
| 2 | Sacramento State | gemma-small-e4b, glm-5 | 4 H200, 1 A6000 |
| 3 | CSU Chico | qwen3-small | 4 A6000 |
| 4 | Fresno State | qwen3-small | 2 H200 |
| 5 | UC San Diego + SDSU | deepseek-v4-flash, gemma, kimi, minimax, qwen3-embedding | 20 A100, 16 H100, 2 L40 |
| 6 | UNM | gemma | 1 A6000 |
| 7 | Nebraska–Lincoln | gemma-small, glm-5, gpt-oss | 9 A10, 8 H100, 4 H200, 2 A6000 |
| 8 | Internet2, Ann Arbor | qwen3 | 8 A100 |

Every pin is a set of model servers in the same cluster, behind the same endpoint. Fresno State and Internet2 Ann Arbor are placed by city.

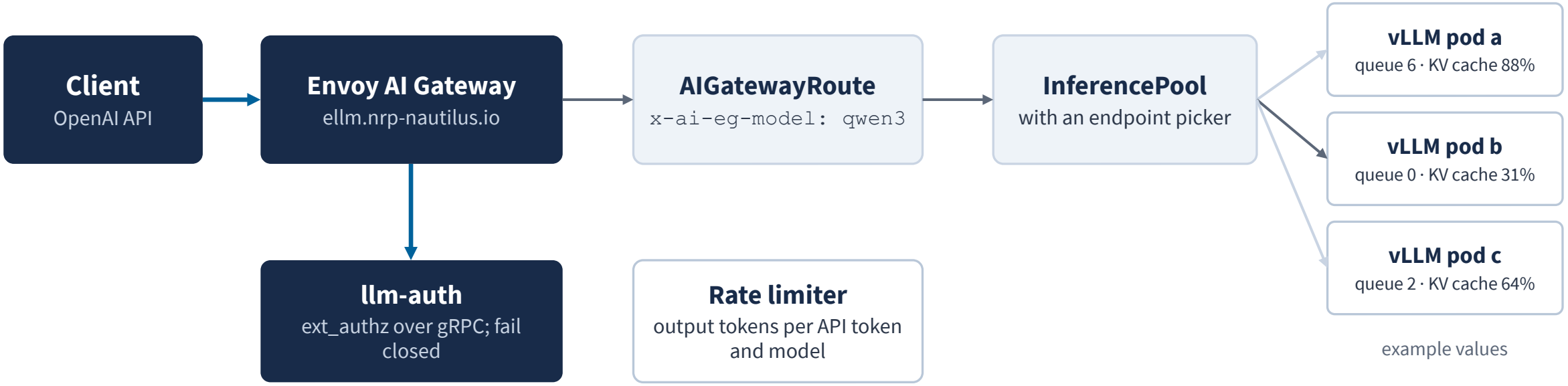
# A Request, Step by Step




## One request, five stops

The gateway is the Envoy Gateway from the ingress slide, plus its AI extension, Envoy AI Gateway. A client asks for `qwen3`.

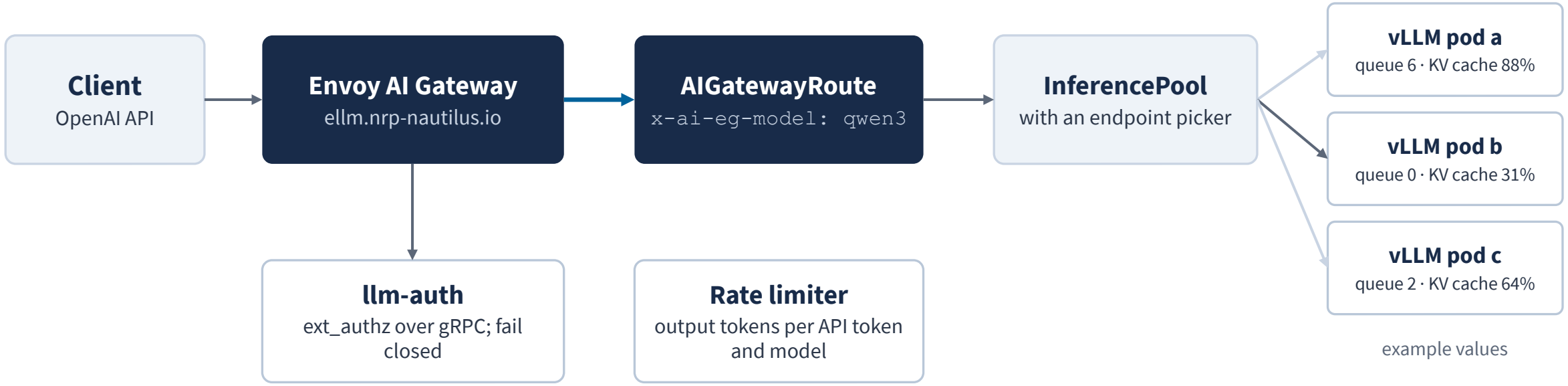
# A Request, Step by Step




## 1. Who are you?

The gateway sends the bearer token to an external auth service over gRPC. If auth can't answer, the request fails: fail closed.

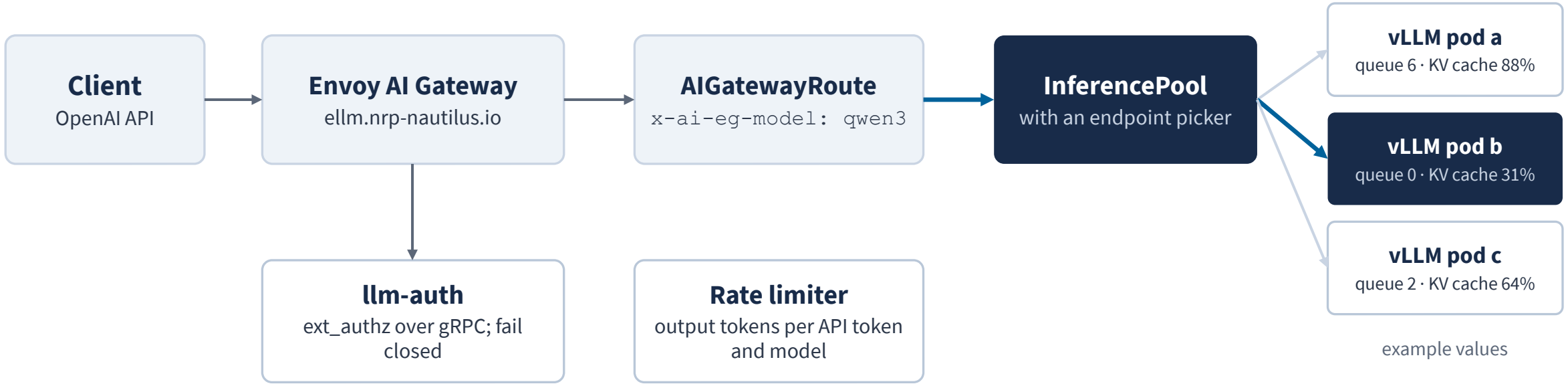
# A Request, Step by Step




## 2. Which model?

The gateway reads `model` from the JSON body into the `x-ai-eg-model` header, and an AIGatewayRoute rule matches it to a backend: here an InferencePool.

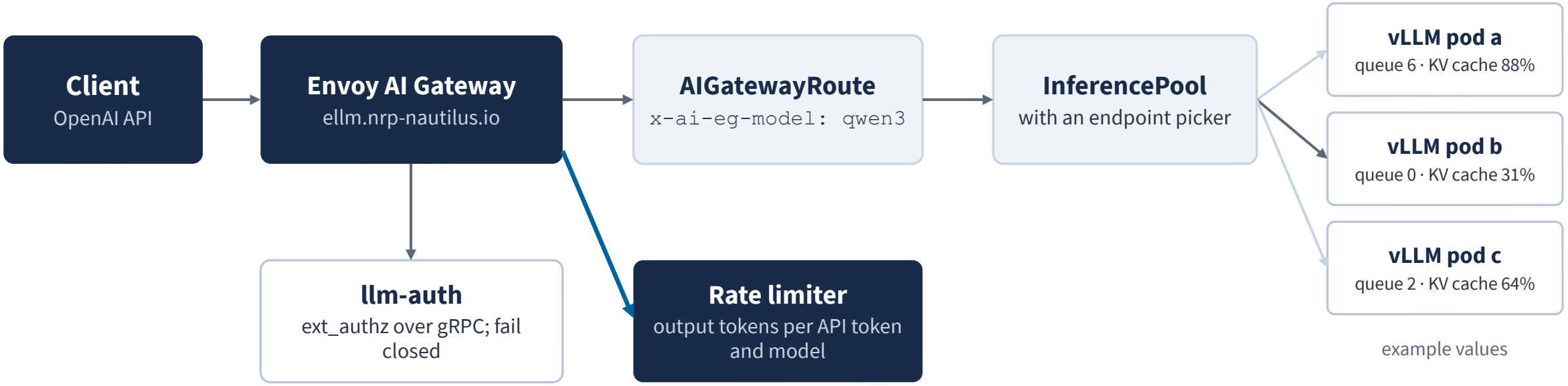
# A Request, Step by Step




## 3. Which pod?

The endpoint picker scores the pool's vLLM pods by queue depth, KV-cache use and prefix-cache hits, and picks one.

# A Request, Step by Step




## 4. The answer, and the token count

vLLM's answer goes back through the gateway, which reads the token counts and tells the rate limiter how many output tokens this API token just used on this model.

# Gateway Policies

```
apiVersion: gateway.envoyproxy.io/v1alpha1
kind: BackendTrafficPolicy
metadata: {name: envoy-ai-gateway-nrp, namespace: nrp-llm}
spec:
  rateLimit:
    type: Global
    global:
      rules:
      - clientSelectors:
        - headers:
          - {name: authorization, type: Distinct}
          - {name: x-ai-eg-model, type: Distinct}
        limit: {requests: 48000, unit: Minute}
        cost:
          request: {from: Number, number: 1}
          response:
            from: Metadata
            metadata: {namespace: io.envoy.ai_gateway,
                       key: llm_output_token}
  retry:
    numRetries: 2
    retryOn: {httpStatusCodes: [500, 502, 503, 504]}
  healthCheck:
    active: {type: HTTP, http: {path: /health}, interval: 10s,
             unhealthyThreshold: 3}
    passive: {consecutive5XxErrors: 2, baseEjectionTime: 30s,
              maxEjectionPercent: 50}
```

**Rate limit:** 48,000 output tokens a minute, per API token and model

**Retries:** twice, on 5xx

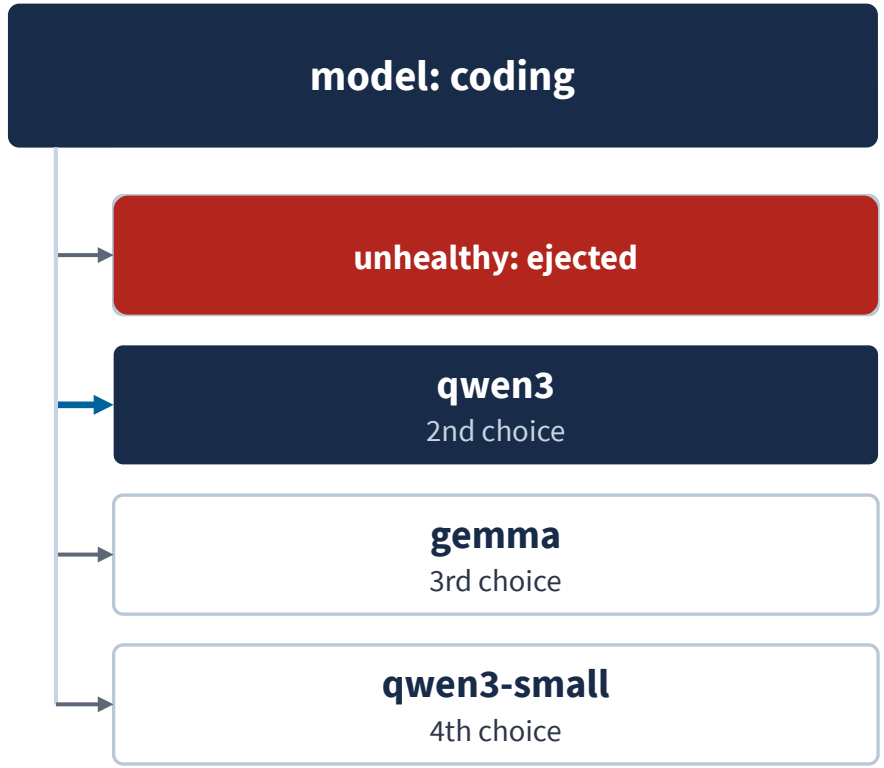
**Health:** probe /health every 10 s; eject a pod after two 5xx

**What a client sees (demo, 10 Nov)**

```
$ curl -si https://ellm.nrp-nautilus.io/v1/chat/completions \
    -H "Authorization: Bearer $OPENAI_API_KEY" \
    -d '{"model": "qwen3", ...}' | grep -i ratelimit
x-ratelimit-limit: 48000, 48000;w=60
x-ratelimit-remaining: 47999
x-ratelimit-reset: 9
```

# Ask for a Task, Not a Model

| Ask for | Tried in this order |
|---|---|
| **agents** | kimi → deepseek-v4-flash → qwen3 → qwen3-small |
| **coding** | deepseek-v4-flash → qwen3 → gemma → qwen3-small |
| **general** | deepseek-v4-flash → qwen3 → qwen3-small → minimax |
| **reasoning** | deepseek-v4-flash → minimax → qwen3-small → qwen3 |
| **vision** | qwen3 → qwen3-small → kimi → deepseek-v4-flash |
| **long-context** | deepseek-v4-flash → gemma → kimi → minimax |
| **kubernetes** | kimi → gemma → qwen3 → qwen3-small |
| **cuda** | deepseek-v4-flash → qwen3-small → gpt-oss → gemma-small-e4b |
| **service** | deepseek-v4-flash → minimax → qwen3 → qwen3-small → gemma → gemma-small → gpt-oss |



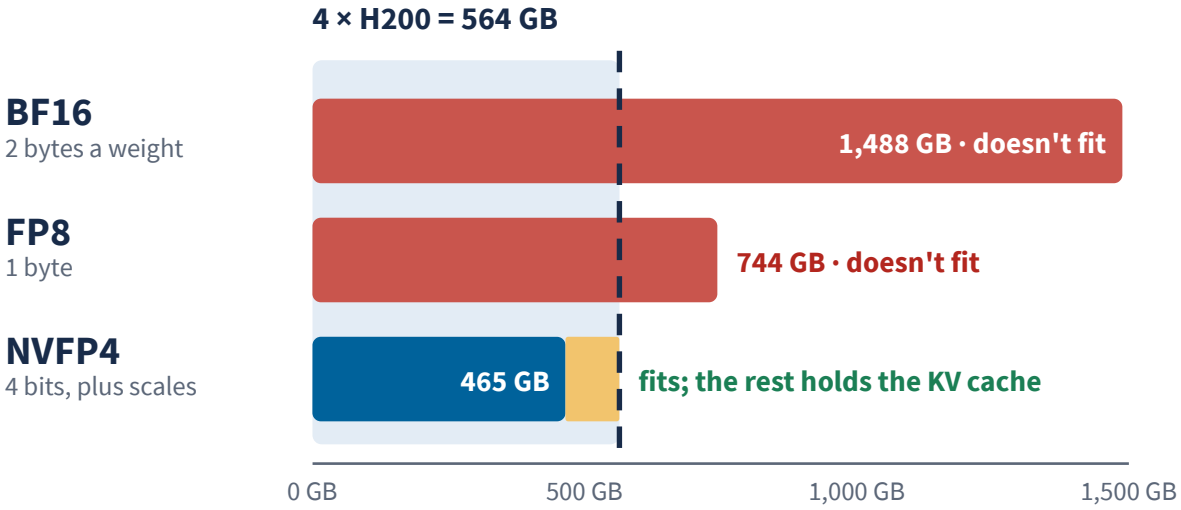

**Only when it's down, or ejected by the health checks, does the 2nd choice answer.**

GitLab Duo Self-Hosted uses `service`.

# Fitting Big Models on Contributed GPUs

## One model: GLM-5.3, 744 B parameters

What its weights need at each precision




- **Quantize:** FP8, NVFP4, MXFP4, INT4: ½ to ¼ the bytes of BF16
- **Split:** tensor parallel inside a node, pipeline parallel across nodes, expert parallel for MoE
- **Budget the KV cache:** what's left after the weights holds the context
- **Place by capability:** GPU memory and generation, not hostname

## What runs where today

| Model | Weights | Format | GPUs |
|---|---|---|---|
| `glm-5` | 465 GB | NVFP4 | 4 × H200, or 8 × H100 |
| `kimi` | 595 GB | INT4 | 8 × A100 |
| `deepseek-v4-flash` | 510 GB | FP8 | 2 nodes × 4 H100 |
| `minimax` | 250 GB | MXFP8 | 8 × A100 |
| `qwen3` | 186 GB | FP8 | 4 × A100 |
| `qwen3-small` | 31 GB | FP8 | 4 × A6000, or 1 × H200 |
| `gpt-oss` | 65 GB | MXFP4 | 2 × A6000 |

Weights: checkpoint size on Hugging Face. GPUs: the model servers on the cluster, 10 Nov 2026.

# Serving with vLLM

qwen3, from the cluster (shortened)

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: qwen38-flash-next-a100, namespace: nrp-llm}
spec:
  replicas: 2
  template:
    metadata:
      labels: {component: llm, service-group: qwen-vllm-inference}
    spec:
      containers:
      - image: vllm/vllm-openai:v0.33.0
        args: ["vllm serve Qwen/Qwen3.8-Flash-Next-FP8 --port 5000
                --tensor-parallel-size 4 --enable-expert-parallel
                --enable-prefix-caching --max-model-len 1000000
                --kv-cache-dtype bfloat16 --otlp-traces-endpoint ..."]
        resources:
          limits: {cpu: "36", memory: 440Gi, nvidia.com/a100: 4}
        readinessProbe: {httpGet: {path: /health, port: 5000}}
      affinity:   # GPU memory > 80 GB, Ampere or newer
  volumeClaimTemplates:
  - metadata: {name: qwen38-flash-cache}
    spec: {storageClassName: linstor-igrok,
           resources: {requests: {storage: 256Gi}}}
```

**Labels** put the pod in the `qwen3` pool, the InferencePool from the request slide

**vLLM**: the FP8 checkpoint, tensor parallel 4, expert parallel, prefix caching, traces to OpenTelemetry

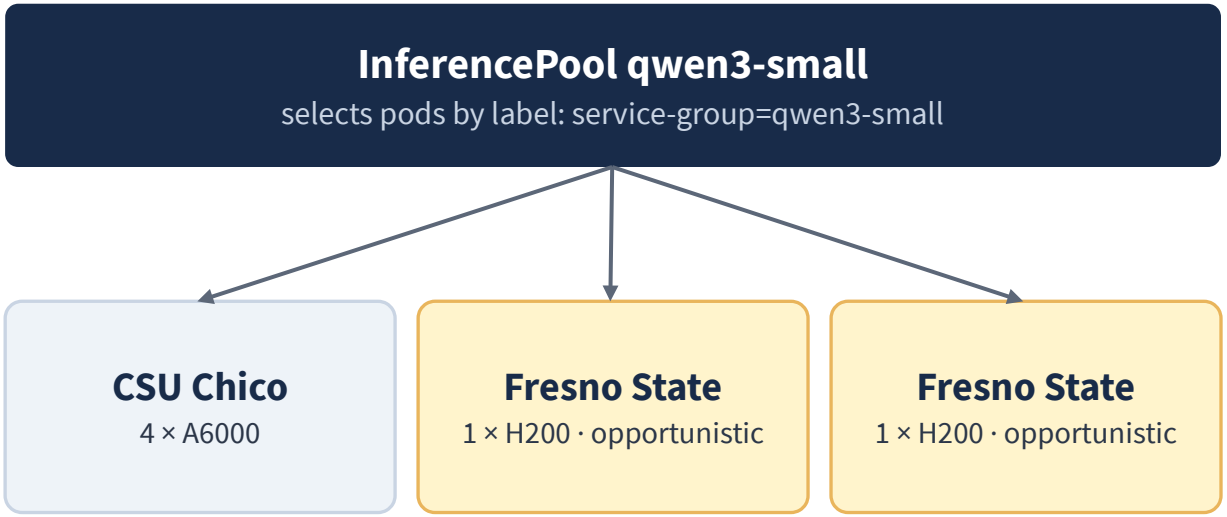
**Health**: probes on `/health`; vLLM and DCGM metrics go to the same Prometheus and Thanos, and drive the LLM status page

**Weights** on a Linstor volume, so a restart doesn't download them again

16 servers like this. Their config is in GitLab (`prp/llm-proxy`), applied by hand, not yet by Argo CD

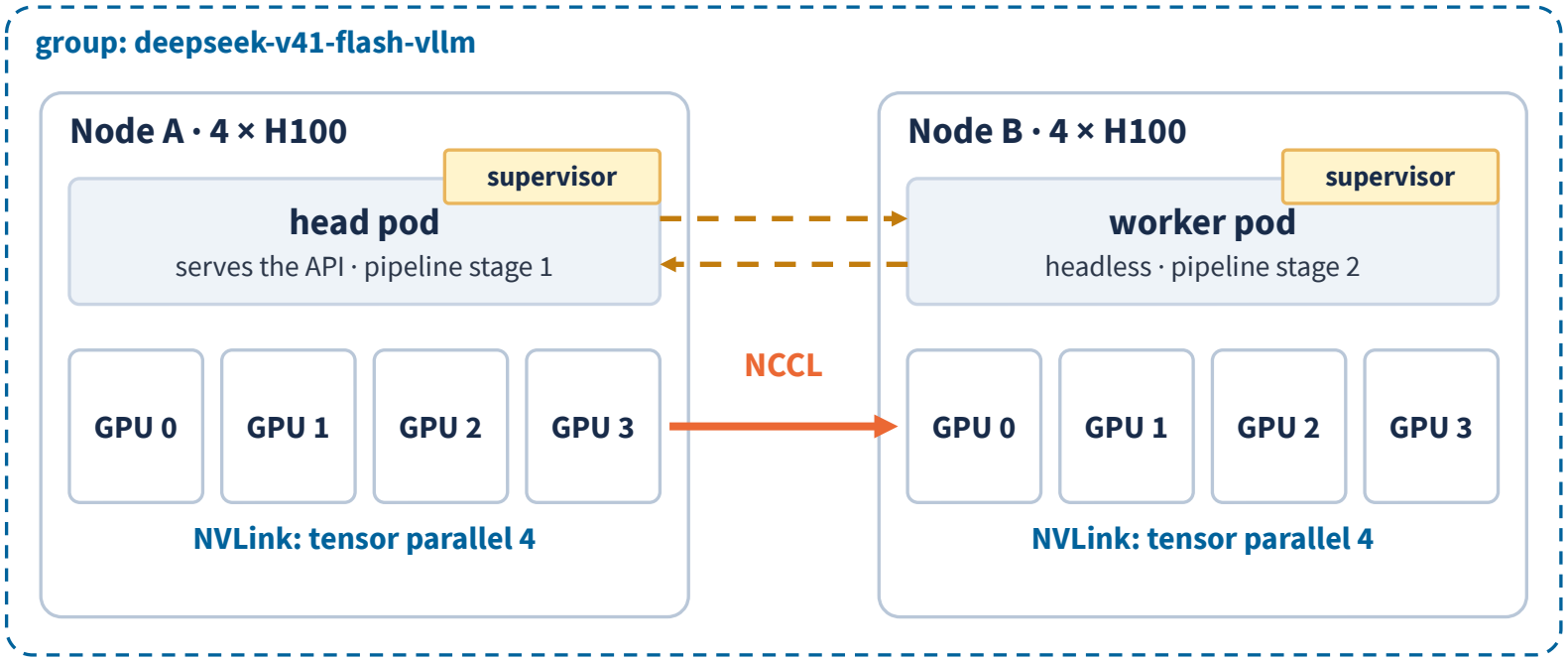
# Borrowing Idle GPUs

- A pool selects its pods by label, so a model can have replicas anywhere
- Extra replicas run at **opportunistic** priority on idle GPUs; on a reserved node they also tolerate its reservation taint
- `qwen3-small`: 4 × A6000 at Chico State, plus two 1-GPU replicas on Fresno State's H200 node, which is reserved for Fresno
- `glm-5` and `gemma` borrow H100s and A100s at low priority too
- When the owners' pods need the GPUs, the borrowed replicas are preempted; the endpoint picker routes around the gap




Fresno's own pods need the GPU back. One replica is preempted; the pool keeps answering from the other two.

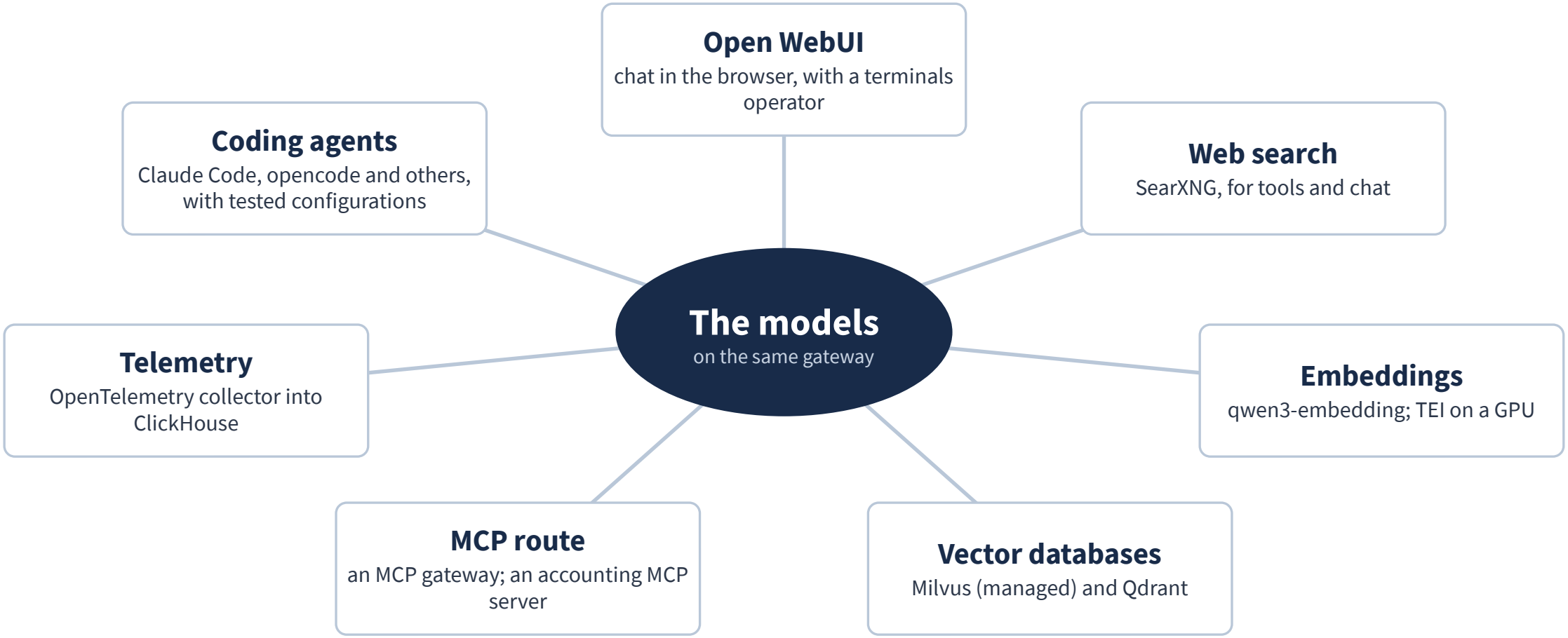
# Multi-Node Serving




Two copies of this pair run on SDSC's Expanse H100 nodes.

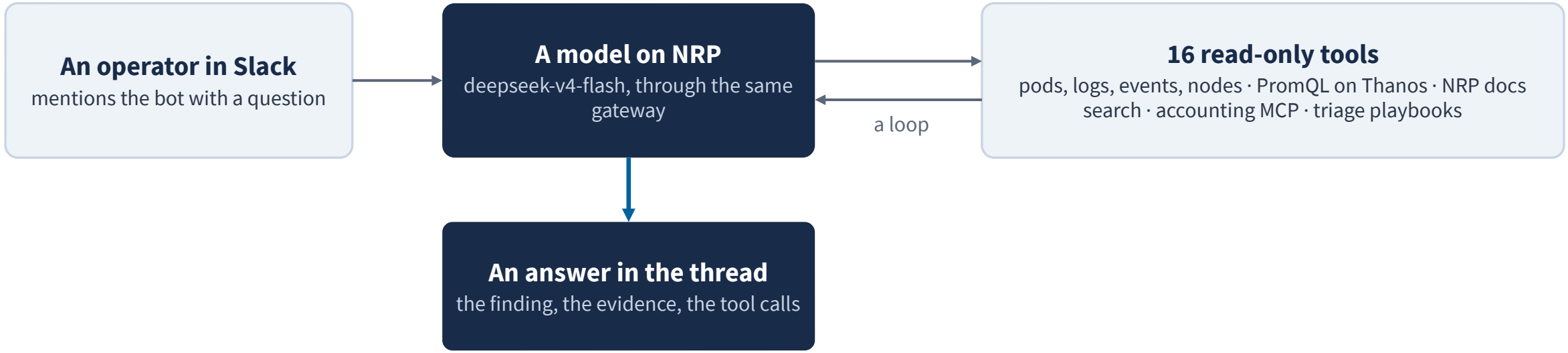
- DeepSeek V4.1 Flash: 510 GB of weights; one 4 × H100 node holds 320 GB
- **Tensor parallel 4** inside each node, over NVLink; the experts spread across the GPUs too
- **Pipeline parallel 2** across the nodes: activations cross the network with NCCL
- Two StatefulSets: the **head** pod serves the API, the **worker** runs headless
- A **supervisor** in each pod restarts both when either restarts (LeaderWorkerSet isn't installed)
- A **headless Service** gives the pods stable names; a **NetworkPolicy** keeps the NCCL traffic to the pair

# Around the Models




All of it on the same cluster, most of it deployed the same way as everything else: from git.

# An LLM on Call: nrp-ops-bot




## Guardrails

- RBAC denies secrets, exec and every write
- Namespace allowlist; credentials redacted
- Logs and events treated as untrusted (prompt injection)
- Budgets on tool calls, iterations and time

## Status

- Phase 1, read-only: built and tested with mocks
- Phase 2, write actions behind two-human approval: not started
- Also an MCP server, and an eval harness to compare models

# Using It Yourself

# Ways In

### JupyterHub
notebooks with a GPU, in the browser

`jupyterhub-west.nrp-nautilus.io`

### Coder
VS Code in the browser, in your namespace

`coder.nrp-nautilus.io`

### kubectl
jobs, services and storage in a namespace

`nrp.ai/config`

### GitLab
repos, CI in Kata VMs, a registry

`gitlab.nrp-nautilus.io`

### LLM API
OpenAI- and Anthropic-compatible

`ellm.nrp-nautilus.io/v1`

### Open WebUI
chat with every model

`nrp-openwebui.nrp-nautilus.io`

Who uses them today: research groups, each in a namespace; courses, each with its own JupyterHub; and collaborations through OSG.

# Getting Access

1. Log in at **nrp.ai**

Log In → CILogon → your institution

2. Get into a namespace

Request one at **get-started.nrp-nautilus.io**

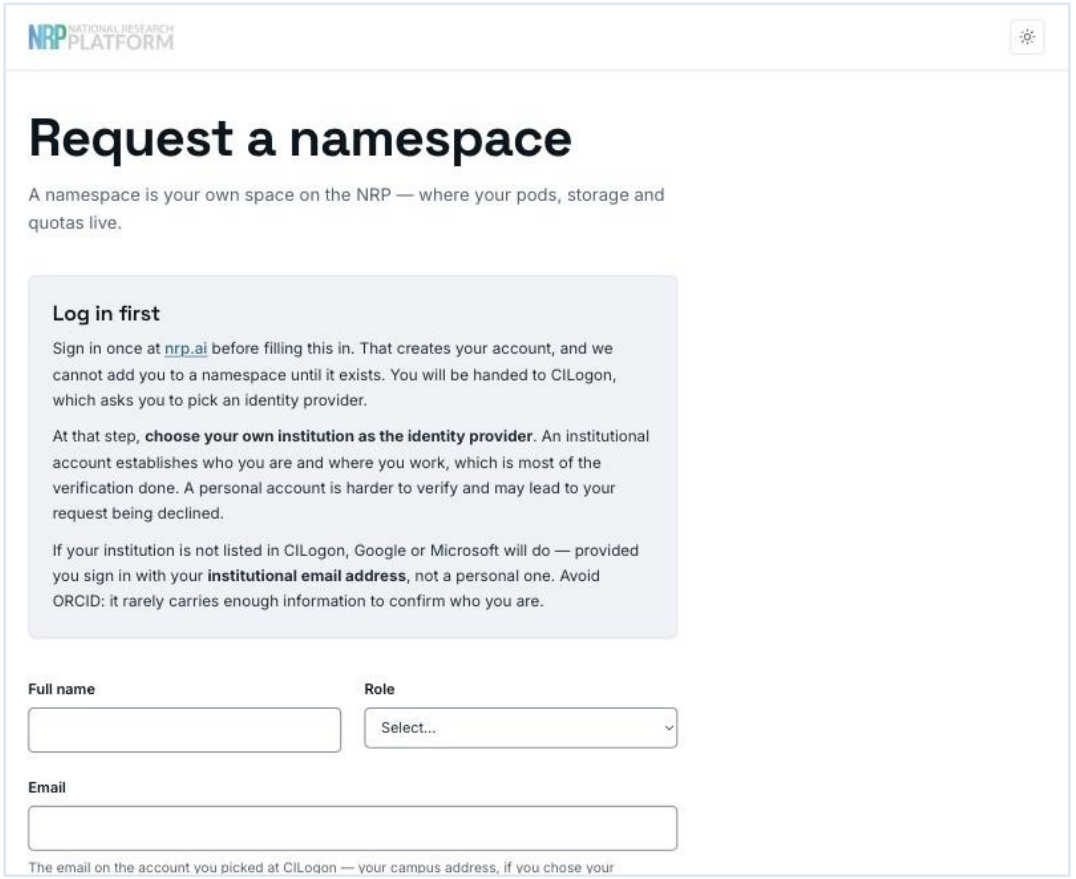
Or ask an existing namespace admin to add you

3. Start working

JupyterHub, kubectl, GitLab, or an LLM token from **nrp.ai/llmtoken**

**Who can request a namespace:** faculty, staff, postdocs and research scientists; students through their PI.

https://get-started.nrp-nautilus.io



# What is a Namespace?

- Your group's space on the cluster: one per lab, project, course or service
- The PI or instructor is its admin and adds members
- Everything you run lives in it: pods, Jobs, volumes, Services, Ingresses
- Members see their own namespaces, not anyone else's
- 2,334 of them today

**Nautilus cluster**

**netlab**
a PI (admin) and three students · GPU jobs · a Ceph volume · a web app

**cse-course**
the instructor (admin) and 40 students · a course JupyterHub · shared datasets

**another-lab**
their members only · you can't see their pods or data

# Ways to Run Work

## Interactive: JupyterHub

- Pick a GPU from the start menu; explore, debug, teach
- Idle servers stop: 1 hour after you close the browser on NRP's hub, 6 hours at most on any hub

## Interactive: a pod

- Start a container with kubectl, then open a shell in it
- Deleted after 6 hours; at most 2 GPUs

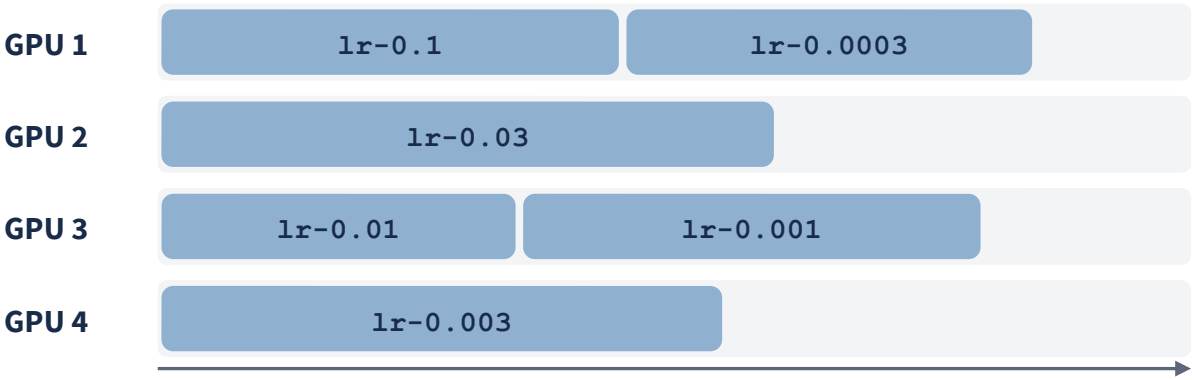
## Long-running: a Deployment

- Keeps a service up (a web app, a database) and restarts it if it crashes
- Removed after 2 weeks unless admins grant an exception; no GPU if it sits mostly idle

## Batch: a Job

- Runs your script to the end, then gives the GPU back
- Submit many at once; the rest wait in Pending for a free GPU

```
$ kubectl apply -f sweep/        # six training jobs
```




Six jobs on four GPUs: two wait in Pending and take the first GPUs that free up. When the last one ends, all four GPUs are free again.

# A GPU Job

```
apiVersion: batch/v1
kind: Job
metadata: {name: gpu-train-daniel, namespace: my-lab}     # (1)
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: train
        image: pytorch/pytorch:2.9.0-cuda13.0-cudnn9-runtime  # (2)
        command: ["python", "train.py"]
        resources:                                          # (3)
          requests: {cpu: "2", memory: 6Gi, nvidia.com/gpu: 1}
          limits:   {cpu: "2", memory: 6Gi, nvidia.com/gpu: 1}
      affinity:                                             # (4)
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - {key: nvidia.com/gpu.memory, operator: Gt, values: ["20000"]}
              - {key: nvidia.com/gpu.compute.major, operator: Gt, values: ["7"]}
```

(1) **Where:** your namespace

(2) **What:** a container image, and the command to run

(3) **Resources:** 2 CPUs, 6 GB of memory, one GPU

(4) **Which GPU:** more than 20 GB, Ampere or newer; or name exact models

# Where to Start

## Use it

- **The LLM API in your automation:** config review, log triage, a docs assistant; task names keep working as the catalog changes
- **Your tools in a namespace:** web apps, Postgres and ClickHouse from the operators, S3 for data
- **GitLab CI** to build and ship them
- **GPUs for AI and ML:** a notebook to start, Jobs when it's real

## Connect to it

- **Contribute nodes**, the way a node joins: you keep priority, NRP runs them
- **InterLink:** a Slurm cluster appears as a virtual node
- **Federation** with another Kubernetes cluster
- **FABRIC and SENSE** testbeds, with Multus for extra NICs
- **perfSONAR:** a testpoint on your node joins the mesh

*https://nrp.ai/documentation/admindocs/participating/new-contributor-guide/*

# Live Demo

# Acknowledgments


- NRP is supported by NSF awards OAC-1541349, OAC-1826967, OAC-2030508, OAC-1841530, OAC-2005369, OAC-2112167, CISE-1713149, CISE-2100237 and CISE-2120019
- Networks: CENIC, Pacific Wave, Internet2, MREN, GPN, NYSERNet, FLR, NEREN, Sun Corridor, OARnet, SCLR and the Albuquerque GigaPoP
- The NRP operations teams at SDSC, the University of Nebraska–Lincoln and MGHPCC, and the campuses that host and share their hardware
- SCinet, for having us during the NOC move

# Questions?

| | |
|---|---|
| Today's materials | `training.nrp-nautilus.io/scinet-sc26` |
| Portal and login | `nrp.ai` |
| Request a namespace | `get-started.nrp-nautilus.io` |
| LLM endpoint | `ellm.nrp-nautilus.io/v1` |
| LLM token | `nrp.ai/llmtoken` |
| perfSONAR | `perfsonar.nrp-nautilus.io` |
| Admin guide | `nrp.ai/documentation/admindocs` |
| Support chat and email | `nrp.ai/contact` |

**Daniel Diaz · d4diaz@ucsd.edu**

# Backup

# What Argo CD Runs

| Application | Source | Sync |
|---|---|---|
| `cve-mitigations` | NRP GitLab: prp/cve-mitigations | auto |
| `falco` | Helm chart: falco | manual |
| `falco-talon` | Helm chart: falco-talon | auto, self-heal |
| `gitlab` | GitHub: nrp-nautilus/gitlab | auto, self-heal |
| `gpu-power-management` | GitHub: Sam6734/nrp-gpu-operator-tracking | auto, self-heal |
| `grafana` | Helm chart: grafana | auto, self-heal |
| `inspect` | NRP GitLab: nrp/inspect | auto, self-heal |
| `kueue` | GitHub: nrp-nautilus/kueue-gitops | manual |
| `kured` | Helm chart: charts | auto |
| `netbox` | Helm chart: netbox | auto, self-heal |

| Application | Source | Sync |
|---|---|---|
| `netbox-agent` | NRP GitLab: nrp/netbox-4 | auto, self-heal |
| `nrp-aime` | GitHub: djw8605/nrp-aime | auto, self-heal |
| `nrp-clickhouse-deploy` | GitHub: djw8605/nrp-clickhouse | auto |
| `nrp-onboarding` | NRP GitLab: prp/nrp-onboarding | auto, self-heal |
| `nrp-ops-bot` | GitHub: djw8605/nrp-ops-bot | auto |
| `openwebui` | NRP GitLab: nrp/openwebui | auto |
| `ray` | NRP GitLab: prp/ray | manual |
| `self-node-remediation` | NRP GitLab: nrp/self-node-remediation | manual |
| `wazuh` | NRP GitLab: nrp/wazuh | auto, self-heal |

Argo CD applications on the cluster, 10 Nov 2026; Argo's own example app left out.

# Runner Flavors

| Runner | For | Privileged | Jobs at once | Limits a job can raise to |
|---|---|---|---|---|
| **`gitlab-runner-kata-default`** | every project on gitlab.nrp-nautilus.io (the default) | no | 10 | 12 CPUs, 50 GiB |
| **`gitlab-runner-kata`** | Docker builds: Docker-in-Docker (`docker:28-dind`) | yes, inside its VM | 3 | Docker gets 48 GiB of memory for image layers |
| **`gitlab-runner-gitlab`** | projects on gitlab.com | no | 2 | 12 CPUs, 24 GiB |
| **`gitlab-runner-sdsc`** | SDSC's own GitLab, hpc-gitlab.sdsc.edu | no | 2 | 12 CPUs, 24 GiB |

- All four run each job in its own **Kata VM**, in namespace `gitlab-ci-kata`, on x86 nodes tainted `nautilus.io/testing=kata`
- Pull policy `always` (except the Docker runner): a job can't start another project's private image from a node's cache
- Groups can also run their own runners in their namespaces, e.g. `ihv-cenic` and `unl-dtih`

# The Agent, Up Close

netbox_agent.yaml (token left out)

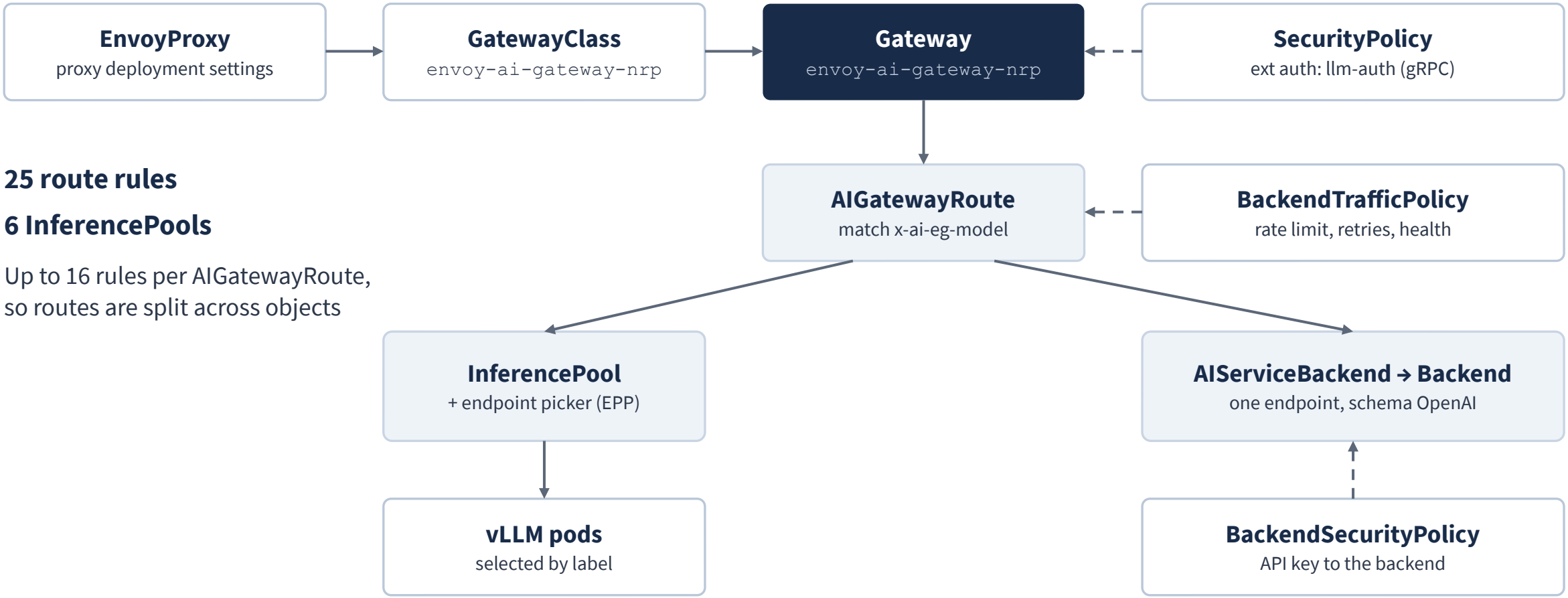
```
netbox:
  url: https://netbox-4.nrp-nautilus.io
network:
  ignore_interfaces: (dummy.*|docker.*|tunl*|cali*|vxlan.*|lo|ibp.*|.*ifb.*)
  ignore_ips: (127\.0\..*)
  lldp: true
  nic_id: name
  primary_mac: permanent
virtual:
  cluster_name: nautilus
datacenter_location:
  driver: cmd:kubectl get node ${NODE_NAME} -o jsonpath=
    '{.metadata.labels.netbox\.io/site}' | tr '[:upper:]' '[:lower:]'
inventory: true
```

The run loop (from the startup script)

```
lldpd           # start listening for LLDP
# at least 30 s so LLDP has found its neighbours
pause $(random_between 30 "$INITIAL_DELAY_MAX")
while true; do
  netbox_agent -c "$RUNTIME_CONFIG" --update-all -d &&
  netbox_agent -c "$RUNTIME_CONFIG" --update-inventory -d
  # a day, plus up to an hour of jitter
  pause $((RUN_INTERVAL + $(random_between 0 "$RUN_JITTER")))
done
```

Interfaces the overlay creates (cali*, vxlan*, tunl*) are ignored, so NetBox holds the physical network only.

# Envoy AI Gateway Objects




**25 route rules**

**6 InferencePools**

Up to 16 rules per AIGatewayRoute, so routes are split across objects

From the NRP admin guide, Managing AI Models; object names from the cluster.

# Using GPUs Fairly

## Do

- Run batch work as a **Job** that exits
- Request what you'll use; keep limits within 20% of requests
- Keep data on a persistent volume or in S3
- Run a sweep as many one-GPU jobs
- Borrow idle A100/H100s with `priorityClassName: opportunistic`, and checkpoint

## Don't

- Leave a notebook holding a GPU overnight
- Put `sleep` in a Job to keep a GPU (bannable)
- Ask for 4 GPUs "just in case": use must stay above 40%
- Keep results on the pod's own disk; it vanishes with the pod

*https://nrp.ai/documentation/userdocs/start/policies/*